

Q&A ed Esempi Large Scale & Multi-structured Databases

Francesco Taverna

Indice

Legenda:	3
Consigli generali	3
Domande teoriche:	4
1. Proprietà ACID & Proprietà BASE	4
2. Cosa succede in caso di network partition? che proprietà vengono perse e mantenute?	5
3. Differenza tra Monotonic, Write e Causal Consistency con contesto.....	8
4. Quando è meglio usare un DB Key value? Perché sono più veloci? Perché vengono usati nei carrelli?	10
5. Nei database Key value, quali sono gli step per estrarre un valore dal database o dalla memoria? Parlami dell'hashing per determinare il nodo di una data chiave e dei conflitti che possono nascere nell'hashing.....	11
6. Parlami delle caratteristiche principali di un key value DB	14
7. XML and Json: pro e contro	16
8. Vantaggi dello schemaless DB.....	18
9. Contesti in cui è meglio usare un documentDB.	19
10. Come decidiamo se due documenti differenti possono andare nella stessa collezione?	19
11. Differenza tra Document embedding e document linking?	20
12. Parlami degli indici in MongoDB. A che livello sono definiti? Cosa sono i Compound Indexes?.....	22
13. Quale architettura è meglio usare per programmare ad oggetti?	24
14. Definizione e differenza tra replicazione e sharding, specificare a cosa servono dal punto di vista delle web scale application.	25
15. Spiegare in cosa consistono replicazione e sharding e a quali requisiti non funzionali portano vantaggi	29
16. Come sharding e partitioning contribuiscono ad avere la availability e in cosa differiscono? Definire le due proprietà.....	29
17. Definizione di Keyspace, Row keys, Column family e Column nei Columnar Database.....	32
18. Bloom Filters	36
19. Anti-entropy process	37

Esercizi pratici.....	40
1. Disegnare tabella vista a lezione dove c'è product, book clothing appliance, definire row key column family e column table, inserire due record con clothing id e uno con solo appliance ID (va disegnata la tabella)	40
2. Relazione one-to-many tra studenti ed esami, disegnare l'UML, definire uno o più requisiti funzionali per una soluzione document embedding e stesso per il document linking, scrivere in JSON i documenti per entrambi i casi	41
3. Esercizio sui document DB: ti do entità e relazioni, fornisci una motivazione per usare il document embedding e una per il document linking, poi abbozzare i documenti nei due casi	44
4. Supponiamo di considerare una relazione one-to-many tra l'entità ristorante e la entità recensioni tale che un ristorante abbia molte recensioni e una recensione sia associata ad un solo ristorante. Discuti con dettagli come questa relazione può essere implementata in un document DB, fornendo un esempio in formato JSON del DB per ognuna delle possibili implementazioni considerando i seguenti dati.	47
5. Considera un namespace, accessibile attraverso la variabile "AppNameSpace" che include le seguenti entrate:	50
6. Esempio delle slide "dal relazionale al keyvalue", ma chiesto col document DB: traduci relazione 1 a m tra Impiegati e Pagamenti nel document DB. Di seguito sono presenti le due tabelle nel modello relazionale, definisci le query e produci il design del document DB in base alle query che hai definito.....	50
7. Esercizio di seguito	52
8. Table product + table books, table appliances, come li traduci in un document DB? In una singola collezione o in molteplici?	53
Esempi dalle slide per la parte di esercizi.....	58
Esempio 1 – Query nel column DB partendo da relational DB	58
Esempio 2 – Evitare le join nelle relazioni many to many Columnar DB	59
Esempio 3 – Modellizzare una entità in una singola riga	60
Esempio 4 – Evitare strutture dati complesse come valori nei columnar	61
Esempio 5 – Indici nei columnar	62
Esempio 6 – Graph DB vs RDB.....	63
Esempio 7 – Many to Many nei Graph DB	64
Esempio 8 – un semplice Graph DB	64
Esempio 9 – Social Network	66
Esempio 10 – Data Modeling.....	68
Esempio 11 – Da relational DB a Key value DB	69
Esempio 12 – Gestire range di valori key value	70
Esempio 13- Valori complessi key value	72

Esempio 14 – Mobile app Key value	73
Esempio 15 – Caso del ristorante.....	74
Esempio 16 – Ecommerce	76
Esempio 17 – Relazioni one to many e many to many document DB	78

Legenda:

- Argomento che probabilmente non chiede
- Domanda sicuramente fatta da quando esiste lo scritto (le altre potrebbero essere state fatte quando esisteva l'orale)
- Domanda che probabilmente non chiede
- Domanda che potrebbe chiedere (chiesta all'orale tempo fa)

Consigli generali

- c'è spesso un esercizio sul columnar, in generale da un UML con relazione 1 a n (può darla anche col document).
- vanno visti tutte le trasformazioni di 1 a m in tutti i DB
- sui key value: implementare 1 a m
- columnar: relazioni 1 a m e n to n
- i disegni devi farli uguali alle slide

Domande teoriche:

Contesto generale:

I più recenti NoSQL DBMS sono utilizzati in sistemi distribuiti, ovvero su molteplici server. I pro di questo approccio:

- Assicurano scalabilità, flessibilità, disponibilità
- Facile aggiungere e rimuovere nodi (grazie alla scalabilità orizzontale) invece che quella verticale
- Aggiungono strategie di fault tolerance

Contro:

- Bisogna bilanciare la consistenza con la disponibilità
- Proteggere dai fallimenti della rete che possono lasciare nodi isolati

1. Proprietà ACID & Proprietà BASE

Risposta:

Contesto: I database in generale permettono di salvare dati e leggerli. I DBMS si occupano di ciò, attraverso 3 principali task:

- Salvare i dati in modo persistente (nessuna perdita quando il database si spegne)
- Mantenere i dati in modo consistente (consistency: tutti i nodi di un sistema vedono gli stessi dati contemporaneamente, se una modifica viene apportata a un dato, tutti i nodi vedono immediatamente il cambiamento)
- Assicurare la disponibilità (availability: i dati devono essere sempre accessibili, bisogna trovare una soluzione ai fallimenti, per esempio avere dei server di backup che intervengono quando uno si rompe)

In generale, queste caratteristiche fanno riferimento ad un contesto distribuito.

Risposta effettiva:

Le transazioni **ACID** (Atomicità, Consistenza, Isolamento, Durabilità) e **BASE** (Basically Available, Soft state, Eventually consistent) rappresentano due approcci distinti per gestire la coerenza e la disponibilità dei dati nei **sistemi distribuiti**.

Nelle transazioni **ACID** si ha:

- **Atomicità:** le transazioni sono indivisibili, ciò significa che o tutti gli statement della transazioni sono applicati al database oppure nessuna lo è.
- **Consistenza:** prima e dopo il completamento di una transazione, il sistema si trova in uno stato consistente.

- **Isolamento:** le transazioni sono isolate le une dalle altre, non vedono gli effetti delle altre "in progress", durante l'esecuzione simultanea, è importante che ogni transazione si comporti come se fosse l'unica in corso, preservando così la consistenza dei dati
- **Durabilità:** una volta che una transazione è salvata nel database, i suoi cambiamenti devono persistere anche se c'è un fallimento nel SO o nell'hardware

Nelle transazioni BASE invece si ha:

- **Basically available:** il sistema garantisce una disponibilità di base, anche a costo della inconsistenza momentanea, resiste a parziali fallimenti del database distribuito grazie alle repliche dei dati.
- **Soft state:** i dati potrebbero non essere aggiornati con quelli più recenti a causa dell'eventual consistency, la scrittura di un utente può non causare un aggiornamento immediato di tutti i nodi
- **Eventually consistent:** la coerenza dei dati viene raggiunta *ad un certo punto nel futuro*, senza garantire un momento specifico in cui i dati saranno coerenti. Il sistema consente ai nodi di convergere gradualmente verso uno stato consistente.

In sintesi, mentre ACID mira a garantire **consistenza immediata e prevedibile** dei dati a scapito della disponibilità in caso di fallimenti, BASE adotta un approccio più rilassato, consentendo una **maggiore disponibilità e tolleranza ai guasti** a costo di una consistenza che può richiedere del tempo per essere raggiunta, in particolare se sono presenti molte repliche da aggiornare, cosa necessaria per garantire disponibilità. Nei database NoSQL spesso si ricorre all'eventual consistency: ci sarà un periodo di tempo dove le copie dei dati hanno valori diversi, ma prima o poi avranno lo stesso valore, quindi in generale si ricorre al BASE.

- Ecommerce: si privilegia la risposta veloce e la disponibilità del sistema, condizioni più importanti che la consistenza su server diversi. Per esempio, il carrello può avere dati non consistenti tra i vari server, ma se uno fallisce la disponibilità è garantita.
- Al contrario però se voglio favorire la consistenza, posso fare ad esempio il two phase commit, il quale però impone che le query sono bloccate durante il periodo di aggiornamento, quindi la disponibilità può non essere garantita sempre.
- Teorema CAP

2. Cosa succede in caso di network partition? che proprietà vengono perse e mantenute?

Risposta:

Contesto: I database in generale permettono di salvare dati e leggerli. I DBMS si occupano di ciò, attraverso 3 principali task:

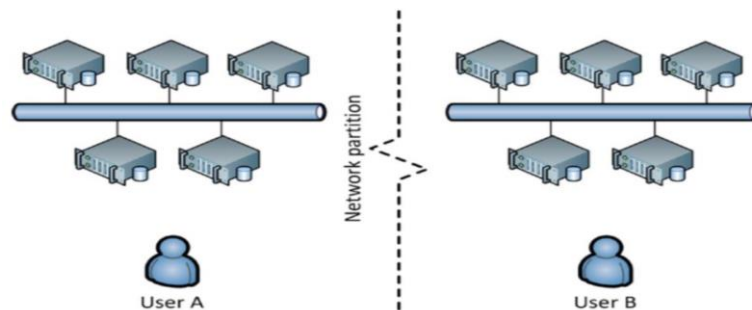
- Salvare i dati in modo persistente (nessuna perdita quando il database si spegne)

- Mantenere i dati in modo consistente (consistency: tutti i nodi di un sistema vedono gli stessi dati contemporaneamente, se una modifica viene apportata a un dato, tutti i nodi vedono immediatamente il cambiamento)
- Assicurare la disponibilità (availability: i dati devono essere sempre accessibili, bisogna trovare una soluzione ai fallimenti, per esempio avere dei server di backup che intervengono quando uno si rompe)

In generale, queste caratteristiche fanno riferimento ad un contesto distribuito.

Risposta effettiva:

Il sistema ha due possibilità: mostrare a ogni utente una diversa visualizzazione dei dati (availability ma non consistency) oppure chiudere una delle partizioni e disconnettere uno degli utenti (consistency ma non availability).

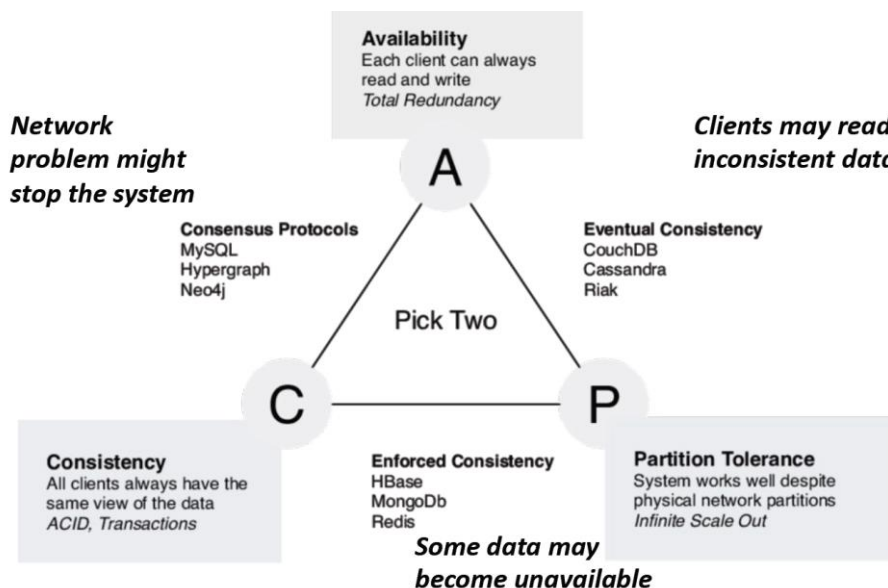


Per capire meglio questo problema introduciamo il teorema CAP.

I database distribuiti non possono garantire allo stesso tempo:

- Consistency (C), la presenza di copie coerenti di dati su server diversi
- Availability (A), ovvero fornire una risposta a qualsiasi query
- Partition protection (P), i guasti dei singoli nodi o le connessioni tra nodi non hanno impatto sul sistema nel suo complesso.

Al massimo **due** delle funzionalità precedenti possono essere trovate in un database distribuito. Lo schema di ciò appena descritto è in figura:



Vediamo le varie soluzioni possibili nel dettaglio:

1. Soluzioni CA:

Nel contesto di un sistema CA, si opta per la **consistency (C)** e la **availability (A)**, ma si è disposti a compromettere la **partition tolerance (P)**. Ciò significa che, in caso di partizione di rete (cioè quando una parte del sistema non può comunicare con un'altra), il sistema **si blocca** per garantire che i dati siano consistenti e disponibili solo quando tutte le parti possono comunicare. Si parla di *Site cluster*, quindi tutti i nodi sono sempre in contatto.

Le soluzioni CA sono più comuni in applicazioni in cui la **consistenza** è fondamentale, come nei sistemi bancari e finanziari, dove è essenziale garantire che le transazioni siano accurate e affidabili. Questi sistemi, ad esempio, sono spesso collegati a database relazionali (RDBMS) che offrono forte consistenza e disponibilità.

Garantire la **consistenza totale** può influenzare negativamente le **prestazioni** del sistema, causando **latenza** e **scalabilità limitata**, perché il sistema deve assicurarsi che tutti i nodi siano aggiornati prima di completare le operazioni, e questo può richiedere tempo.

2. Soluzioni AP

In un sistema distribuito **AP**, è garantita la **availability** e la **partition tolerance**. Tuttavia, durante una partizione di rete, i dati restituiti possono essere **inaccurati**.

Caratteristiche principali:

- **Disponibilità (A):** Il sistema continua a rispondere alle richieste, anche se non garantisce che i dati siano aggiornati o coerenti.
- **Tolleranza alla partizione (P):** Il sistema è in grado di funzionare anche se alcune parti non possono comunicare tra loro.
- **Compromesso sulla consistenza (C):** In questi sistemi, la consistenza dei dati è sacrificata in favore della disponibilità e della tolleranza alla partizione. Ciò significa che il sistema può restituire la versione più recente di un dato che potrebbe essere obsoleta o non sincronizzata con il resto dei nodi.

Note importanti:

- **Dati obsoleti:** Un sistema AP può ritornare la versione più recente dei dati che potrebbe essere "stale" (obsoleta). Può anche accettare scritture che verranno elaborate successivamente, quando la partizione è risolta.
- **Esempi di utilizzo:** Sistemi in cui è essenziale garantire la disponibilità continua, anche in caso di errori esterni, come **carrelli della spesa online** e **sistemi di gestione dei contenuti (CMS) per la pubblicazione di notizie**.
- **Considerazioni:**

Le soluzioni AP sono ideali per applicazioni dove la disponibilità è più importante della consistenza immediata, accettando la possibilità di dati temporaneamente non aggiornati.

3. Soluzioni CP

Le soluzioni **CP** garantiscono la **consistenza** e la **tolleranza alla partizione**. In altre parole, il sistema è in grado di mantenere la coerenza dei dati anche durante la divisione della rete.

Tuttavia, durante una partizione, il sistema potrebbe non essere sempre disponibile per rispondere alle richieste.

Caratteristiche principali:

- **Consistenza (C):** Il sistema assicura che tutte le copie dei dati siano sempre coerenti e aggiornate, quindi, dopo una scrittura, tutte le letture restituiranno la versione più recente dei dati.
- **Tolleranza alla partizione (P):** Il sistema può continuare a funzionare anche quando ci sono problemi di comunicazione tra nodi.

Applicazioni ideali:

Le soluzioni CP sono adatte a **applicazioni che richiedono coerenza** e in cui un po' di latenza è accettabile. Un esempio comune è il **sistema ATM bancario**, dove è fondamentale che i dati siano aggiornati e coerenti, anche se ciò implica un certo ritardo nella risposta.

Tecnologie tipiche:

Le soluzioni CP si basano su sistemi distribuiti e replicati, sia **relazionali** sia **NoSQL**, che sono progettati per supportare la consistenza e la tolleranza alla partizione.

Parametri di configurazione:

Molti dei framework NoSQL moderni permettono di configurare il livello di **disponibilità** secondo parametri specifici, offrendo la possibilità di bilanciare le esigenze di consistenza e disponibilità in base ai requisiti dell'applicazione.

Considerazioni:

- **Scelta in base all'analisi dei requisiti:** È importante selezionare soluzioni CP solo dopo aver effettuato un'analisi approfondita dei requisiti del sistema,

3. Differenza tra Monotonic, Write e Causal Consistency con contesto.

Risposta:

Contesto: I più recenti NoSQL DBMS sono utilizzati in sistemi distribuiti, ovvero su molteplici server, nei quali si hanno più repliche degli stessi servizi. Tra i pro di questo approccio si ha scalabilità, flessibilità, disponibilità, ma quest'ultima deve essere bilanciata con la consistenza. Per consistency, appunto, si intende che tutti i nodi di un sistema vedono gli stessi dati contemporaneamente, ovvero, se una modifica viene apportata a un dato, tutti i nodi vedono immediatamente il cambiamento attraverso l'invio di aggiornamenti. Mentre con disponibilità (availability) si intende che i dati devono essere sempre accessibili, dunque bisogna trovare una soluzione ai fallimenti, per esempio avere dei server di backup che intervengono quando uno si rompe.

Risposta effettiva: Nei database NoSQL spesso si ricorre all'eventual consistency: ci sarà un periodo di tempo dove le copie dei dati hanno valori diversi, ma prima o poi avranno lo stesso valore. Ci sono vari tipi di consistenza, tra le quali abbiamo:

- **monotonic consistency**: si divide in *monotonic write* e *monotonic read consistency*. La write assicura che, se un utente effettua vari aggiornamenti, essi siano eseguiti nell'ordine in cui li ha inviati. La read assicura che, se un utente effettua una query e vede un risultato, nessun'altro utente vedrà mai una versione più vecchia di quel valore.

Esempio read: assumiamo che Alice aggiorni il saldo in sospeso di un customer. Ora il saldo è 1500 euro. Lei lo aggiorna a 2500 euro. Bob legge il saldo e vede 2500. Se Bob legge nuovamente il saldo, vedrà sempre 2500 anche se non tutti le copie hanno questo valore aggiornato.

Esempio write: se un utente ha il saldo in sospeso (da pagare) a 1000 euro e Alice lo riduce del 10%, il saldo è 900 euro. Quell'utente spende 1100 euro, quindi il suo saldo sarà di 2000 euro. Ora però se considero le operazioni in ordine differente ottengo un risultato diverso: se prima l'utente spende, avrà 2100 euro di saldo, poi Alice lo riduce e lo sconto sarà di 210 euro rispetto ai 100 precedenti.

- **read-your-write consistency**: assicura che, se un utente aggiorna un record, tutte le sue letture daranno come risultato il record aggiornato

Esempio: Alice aggiorna il saldo di un cliente a 1500 euro. L'aggiornamento è scritto su un server, il processo di replica inizia ad aggiornare le altre copie. Durante la replicazione Alice legge il saldo, in questo caso si ha garantito che legga 1500 se questo tipo di consistenza è presente.

- **causal consistency**: assicura che gli utenti vedano i risultati in modo consistente con le relazioni causali. Con relazione causale si intende che una operazione dipende causalmente da una operazione precedente.

Esempio: immaginiamo un social network con commenti e risposte

Luca: ho passato l'esame!

Francesco: complimenti!

Marco: da me c'è il sole!

Francesco: che fortuna da me piove

Francesco: ho pubblicato un articolo.

Posso vedere i commenti di Luca, Marco e Francesco con ordine differente, ma non posso vedere le risposte in ordine differente, come ad esempio:

Francesco: ho pubblicato un articolo.

Francesco: che fortuna da me piove

Francesco: complimenti!

Marco:

Luca:

4. Quando è meglio usare un DB Key value? Perché sono più veloci? Perché vengono usati nei carrelli?

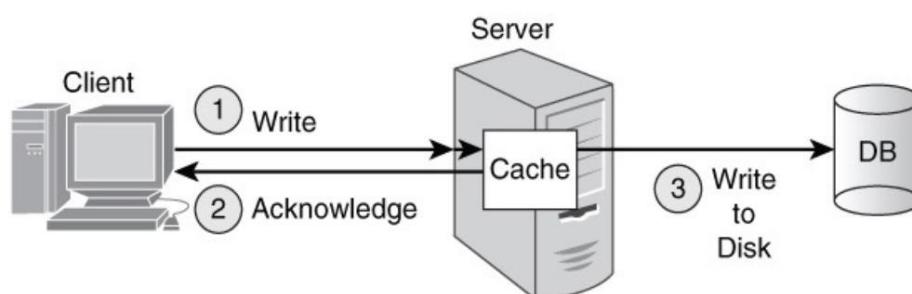
Risposta:

Contesto: I Key value DB si basano sul concetto di array associativi, i quali generalizzano l'idea di una lista ordinata. L'indice in questo array non è più un intero ma può essere un identificatore generico come una stringa, un intero ecc. I valori invece possono essere di tipi differenti (esempio: "foo", 123...). L'unico requisito è che ogni valore ha un identificatore unico sotto forma di chiave. Le chiavi devono essere uniche all'interno di un namespace definito da un bucket specifico.

Bucket 1		Bucket 2	
'Foo1'	'Bar'	'Foo1'	'Baz'
'Foo2'	'Bar2'	'Foo4'	'Baz3'
'Foo3'	'Bar7'	'Foo6'	'Baz2'

Risposta effettiva: è meglio usarli se siamo più interessati alle performance (alta disponibilità, breve tempo di risposta...) e/o i dati non sono troppo complicati (nessuna organizzazione gerarchica, limitato numero di relazioni). Sono molto semplici e facili da gestire, i dati possono essere modellizzati in maniera più semplice rispetto ad un database relazionale dato che non c'è uno schema definito e nemmeno tipi di attributi, oltre che è facilmente modificabile il codice per aggiungere nuovi attributi.

La velocità, come detto, è un punto di forza: infatti il KV può scrivere i dati recentemente aggiornati sul disco, mentre il programma utente sta facendo altro.



Le read sono più facili se i dati sono salvati in memoria. Per questo si usa la cache, infatti la prima volta che il programma prendi i dati, leggerà dal disco ma dopo i dati sono salvati in memoria. Qualora la memoria RAM disponibile finisca, il database deve liberarne un po', per fare ciò sono usati diversi algoritmi per selezionare i record da eliminare, come ad esempio l'LRU, ovvero i dati letti/scritti non di recente quanto gli altri possono essere eliminati.

Possono essere usati nei carrelli perché hanno dati volatili, dove si hanno aggiornamenti frequenti e quindi:

- gestisce la scalabilità orizzontale, quindi milioni di clienti con milioni di carrelli in continuo aggiornamento possono essere gestiti
- alte performance per leggere velocemente il carrello dalla memoria, magari salvato in una unica chiave
- alta disponibilità per permettere una lettura veloce anche in caso di guasti

5. Nei database Key value, quali sono gli step per estrarre un valore dal database o dalla memoria? Parlami dell'hashing per determinare il nodo di una data chiave e dei conflitti che possono nascere nell'hashing.

Risposta:

Contesto: È importante usare una chiave con un significato rispetto a quelle senza nei database relazionali, ma non bisogna scegliere chiavi con un significato troppo specifico per non perdere flessibilità. Un valore è associato solamente alla sua chiave, la quale è una sorta di "indirizzo" della locazione dove quel valore è salvato, deve essere unica nel namespace. Possiamo incorporare informazioni riguardo a vari attributi di una entità nelle chiavi. Per esempio, si può definire un singolo namespace per le informazioni di un customer con la seguente struttura della chiave:

NomeEntità:IdEntità:AttributoEntità → customer:1231:firstName

Key
customer:1982737: firstName
customer:1982737: lastName
customer:1982737: shippingAddress
customer:1982737: shippingCity
customer:1982737: shippingState
customer:1982737: shippingZip

Quindi salviamo uno specifico valore per ogni chiave. I valori sono oggetti e possono essere di qualsiasi tipo (stringhe, liste o oggetti più complessi ...) mentre le chiavi sono stringhe di caratteri. Le chiavi in genere non devono essere troppo lunghe sennò occuperebbero troppa memoria, dato che già sono database *memory intensive*, ma neanche troppo corte sennò possono nascere collisioni.

Il namespace è una collezione di coppie chiave valore che non hanno chiavi duplicate, ma ovviamente si possono avere valori duplicati. I namespace quindi permettono di avere chiavi duplicate ma senza avere conflitti, avendo le collezioni di chiavi separate. Sono utili quando molteplici applicazioni usano lo stesso database key value.

Risposta effettiva: Le operazioni permesse in un KV DB sono: prendere un valore data una chiave (get), inserire un valore data la chiave (set), eliminare un valore data la chiave (delete).

```
define getCustAttr(p_id, p_attrName)
    v_key = 'cust' + ':' + p_id + ':' + p_attrName;
    return (AppNameSpace[v_key] );
```

```
define setCustAttr(p_id, p_attrName, p_value)
    v_key = 'cust' + ':' + p_id + ':' + p_attrName
    AppNameSpace[v_key] = p_value
```

AppNameSpace is the name of the ***namespace*** holding keys and values for this application.

La set e la get migliorano la leggibilità del codice riducendo l'uso di operazioni a basso livello.

Se volessi effettuare ricerche più complicate come ad esempio “tutti gli indirizzi in cui la città è Sarzana” devo farlo nel programma (si leggono ad esempio tutte le chiavi e si salvano le stringhe dove compare quella città). Vedi esempio seguente:

Complex Queries with KV Database

```
appData[cust:9877:address] = '1232 NE River Ave, St.
Louis, MO'
```

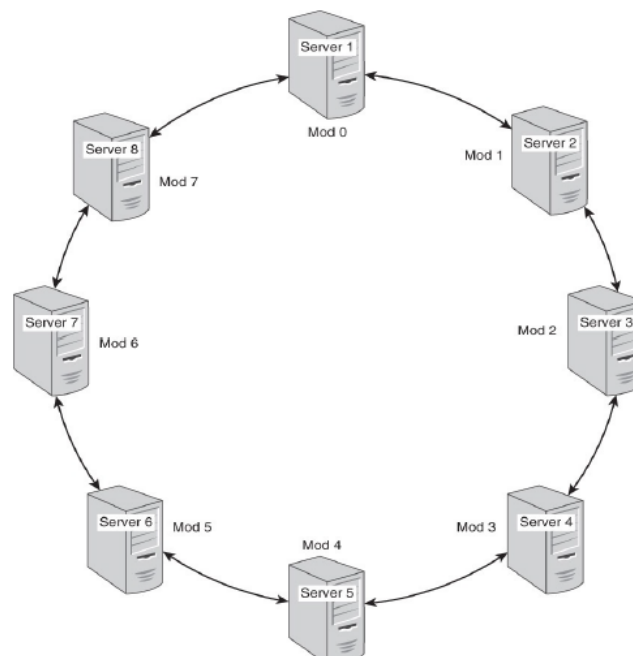
```
define findCustomerWithCity(p_startID, p_endID, p_City):
    begin
        # first, create an empty list variable to hold all
        # addresses that match the city name
        returnList = ();
        # loop through a range of identifiers and build keys
        # to look up address values then test each address
        # using the inString function to see if the city name
        # passed in the p_City parameter is in the address
        # string. If it is, add it to the list of addresses
        # to return
        for id in p_startID to p_endID:
            address = appData['cust:' + id + ':address'];
            if inString(p_City, Address):
                addToList(Address,returnList );
        # after checking all addresses in the ranges specified
        # by the start and end ID return the list of addresses
        # with the specified city name.
        return(returnList);
end;
```

Le funzioni hash sono usate spesso per ottenere *l'indirizzo* della specifica locazione dove il valore è salvato. Esse ritornano un numero che sembra essere randomico partendo da una chiave arbitraria. Usare le hash però implica che possono nascere delle collisioni, perché i valori ritornati da esse possono essere non unici e collidere. In generale, il formato del valore ritornato è esadecimale.

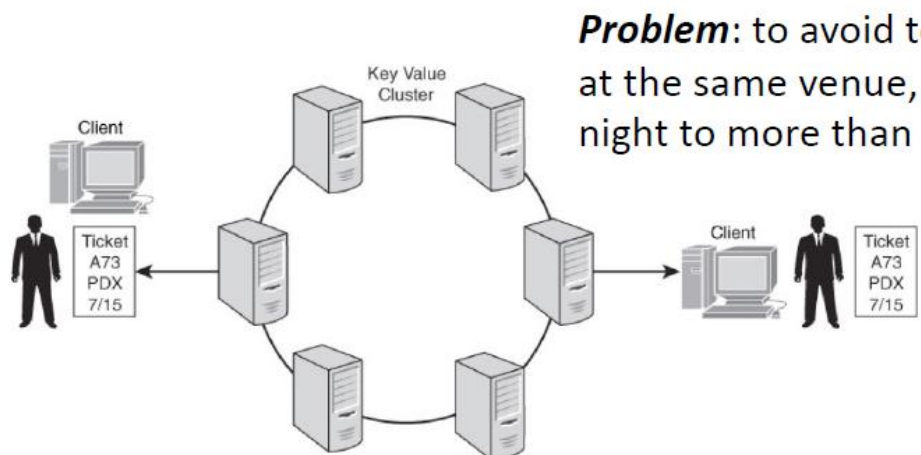
Hash Mapping

Key	Hash Value
customer:1982737: firstName	e135e850b892348a4e516cfcb385eba3bfb6d209
customer:1982737: lastName	f584667c5938571996379f256b8c82d2f5e0f62f
customer:1982737: shippingAddress	d891f26dcdb3136ea76092b1a70bc324c424ae1e
customer:1982737: shippingCity	33522192da50ea66bfc05b74d1315778b6369ec5
customer:1982737: shippingState	239ba0b4c437368ef2b16ecf58c62b5e6409722f
customer:1982737: shippingZip	814f3b2281e49941e1e7a03b223da28a8e0762ff

Per bilanciare il carico in scrittura nei server della masterless replication: immaginiamoci che ogni server ha un ID che lo indirizza, se dividiamo il valore hash per il numero di server possiamo considerare il resto, cioè il modulo. Se abbiamo 8 server, il modulo va da 0 a 7, possiamo assegnare un numero tra 1 e 8 ad ogni server (server 1 – mod 0...).



Per esempio, se due persone differenti volessero comprare lo stesso identico biglietto (stessa data, stesso posto a sedere...) esso avrà una ben specifica chiave, ma essendoci più persone che vogliono comprarlo ci sarà la stessa chiave per entrambi; quindi, si può trasformare con l'hashing + modulo che risulterà nello stesso indirizzo server, ovvero le richieste vengono indirizzate solo a quel server che venderà il biglietto ad uno dei due!

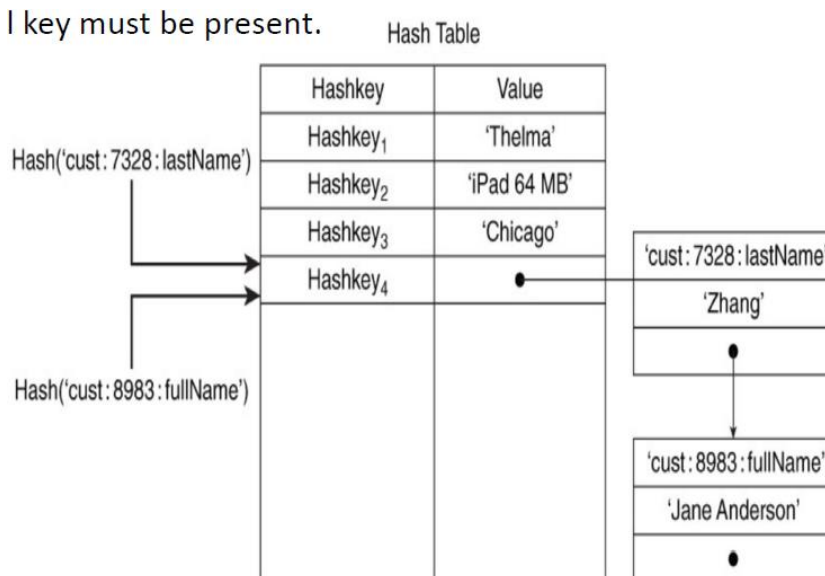


Quindi nessun altro server può vendere quel biglietto.

Inoltre, le hash possono identificare lo specifico cluster (come già detto) o la specifica partizione. Il valore può essere salvato in una partizione, essa è identificata dalla partition key. Ogni chiave in un key value è usata come una partition key.

Infine, le hash sono anche sensibili a piccole variazioni dell'input, quindi permettono di distribuire equamente le chiavi nel range di possibili output. Per esempio, si può prendere il primo carattere della stringa esadecimale generata dalla hash e considerarlo il numero di partizione a cui assegnare quella specifica chiave. Per risolvere le collisioni, si possono usare invece che valori, delle liste legate alla hashing key, nelle quali ogni elemento ha due valori: chiave originale (stringa) e il valore.

I key must be present.



6. Parlatemi delle caratteristiche principali di un key value DB

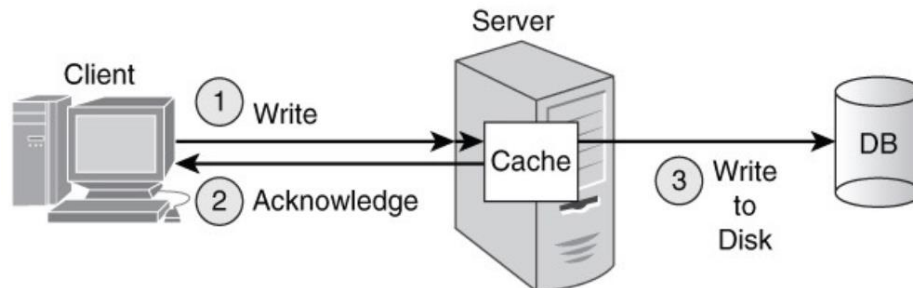
Risposta:

Contesto: I Key value DB si basano sul concetto di array associativi, i quali generalizzano l'idea di una lista ordinata. L'indice in questo array non è più un intero ma può essere un identificatore generico come una stringa, un intero ecc. I valori invece possono essere di tipi

differenti (esempio: "foo", 123). Nei key value i dati sono salvati in maniera persistente, non abbiamo bisogno di tabelle se non abbiamo bisogno di gruppi legati agli attributi. L'unico requisito è che ogni valore ha un identificatore unico sotto forma di chiave. Le chiavi devono essere uniche all'interno di un namespace definito da un bucket specifico (disegno bucket tabella).

Risposta effettiva: Le caratteristiche essenziali di un key value DB sono:

- la scalabilità: Con scalabilità si intende la capacità di aggiungere o rimuovere i server da un cluster quando è necessario per gestire il carico del sistema, bisogna gestire correttamente le letture e le scritture quando il sistema deve essere scalato (in generale si usano due approcci: master slave replication e masterless replication). Sono principalmente usati quando si vuole facilità di salvataggio e di recupero dei dati, rispetto invece a organizzare i dati in strutture più complesse come le tabelle.
- la velocità: il KV può scrivere i dati recentemente aggiornati sul disco, mentre il programma utente sta facendo altro.



Le read sono più facili se i dati sono salvati in memoria. Per questo si usa la cache, infatti la prima volta che il programma prende i dati, leggerà dal disco ma dopo i dati sono salvati in memoria. Qualora la memoria RAM disponibile finisca, il database deve liberarne un po', per fare ciò sono usati diversi algoritmi per selezionare i record da eliminare, come ad esempio l'LRU, ovvero i dati letti/scritti non di recente quanto gli altri possono essere eliminati.

- la semplicità: Se non abbiamo bisogno delle caratteristiche dei modelli relazionali (tabelle di join o query su molteplici entità) può essere una buona idea sfruttare i KV. Molto semplice perché non bisogna definire uno schema (schema-less) e nemmeno i tipi di ogni attributo. Basta modificare il codice per gestire nuovi attributi senza la necessità di modificare il database. Per quanto riguarda l'esempio nella tabella di seguito, potremmo decidere che archiviare il nome completo di un cliente in un singolo valore sia una cattiva idea, quindi separeremo nome e cognome.

Key-Value Database	
Keys	Values
cust:8983:firstName	'Jane'
cust:8983:lastName	'Anderson'
cust:8983:fullName	'Jane Anderson'

- Sono molto flessibili perché anche se facciamo un errore nell'assegnare un tipo di dato sbagliato ad un attributo o usiamo differenti tipi di dati per uno specifico attributo non viene restituito un errore.

I KV DB possono essere usati per salvare i prodotti in un carrello online.

- Indexes
- Consistent hashing

7. XML and Json: pro e contro

Risposta:

Contesto: XML è un markup language, ovvero un linguaggio che specifica la struttura e il contenuto di un documento. Sono aggiunti dei tag per aggiungere informazioni aggiuntive per permettere di capire il significato dei dati.

```

<item>
  <title>Kind of Blue</title>
  <artist>Miles Davis</artist>
  <tracks>
    <track length="9:22">So What</track>
    <track length="9:46">Freddie Freeloader</track>
    <track length="5:37">Blue in Green</track>
    <track length="11:33">All Blues</track>
    <track length="9:26">Flamenco Sketches</track>
  </tracks>
</item>
<item>
  <title>Cookin'</title>
  <artist>Miles Davis</artist>
  <tracks>
    <track length="5:57">My Funny Valentine</track>
    <track length="9:53">Blues by Five</track>
    <track length="4:22">Airegin</track>
    <track length="13:03">Tune-Up</track>
  </tracks>
</item>
<item>
  <title>Blue Train</title>
  <artist>John Coltrane</artist>
  <tracks>
    <track length="10:39">Blue Train</track>
    <track length="9:06">Moment's Notice</track>
    <track length="7:11">Locomotion</track>
    <track length="7:55">I'm Old Fashioned</track>
    <track length="7:03">Lazy Bird</track>
  </tracks>
</item>

```

I JSON sono usati per formattare i dati, usati per lo scambio di dati in internet grazie all'integrazione con JavaScript. JSON risulta una modalità indipendente per specificare gli oggetti come coppie nome-valore, infatti gli oggetti sono un insieme non ordinato di coppie nome-valore. Un array invece è una collezione ordinata di valori, esso può contenere anche documenti, permettendo una struttura gerarchica complessa. I valori possono essere innestati, anche un oggetto o un array possono essere un valore. La lista di coppie (un oggetto) sta tra {} mentre le coppie sono divise da "," e c'è un ":" tra nome e valore, come nella figura seguente. Gli array invece sono tra [].

```

{"skillz": {
  "web": [
    {"name": "html",
      "years": "5"},
    {"name": "css",
      "years": "3"},
  ],
  "database": [
    {"name": "sql",
      "years": "7"},
  ]
}}

```

Risposta effettiva: I vantaggi di XML sono:

- rappresentare qualsiasi tipo di informazione
- sintassi semplice
- leggibile dall'uomo
- è basato sul testo quindi può essere trasmesso efficientemente
- un documento XML può essere mostrato in modo diverso in base alle piattaforme software
- può anche essere modularizzato, quindi alcune parti possono essere riusate

Gli svantaggi di XML sono:

- i tag sono verbosi e ripetitivi, quindi l'ammontare di memoria richiesta incrementa

- i documenti sprecano spazio e l'analisi è costosa a livello computazionale

In generale gli XML DB sono usati per sistemi di gestione di contenuti come collezioni di file di testo, organizzati e mantenuti in formato XML.

I JSON DB sono più usati per i carichi di lavoro basati sul web come memorizzazione e modifica di contenuti dinamici. Questi DB hanno un documento come unità base di memorizzazione.

Vantaggi dei JSON:

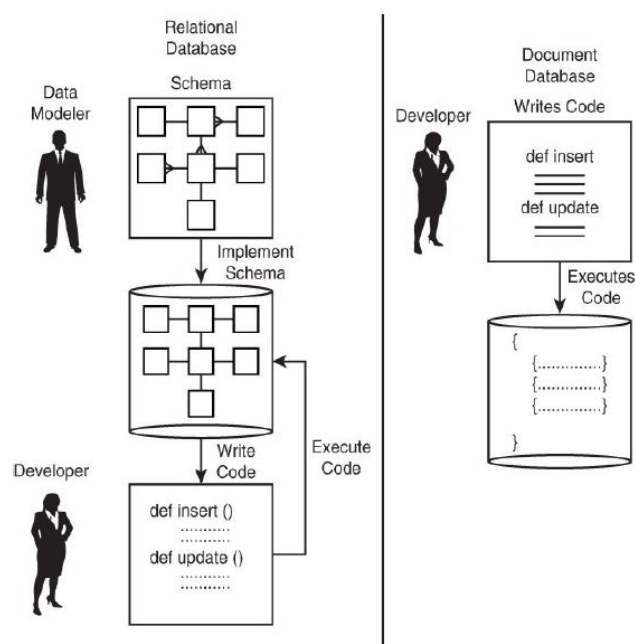
- sintassi semplice
- leggibile dall'uomo
- meno verboso
- include gli array
- il suo database è schemaless

8. Vantaggi dello schemaless DB

Risposta:

Contesto: I document DB, in generale, sono schemeless, come ad esempio il JSON DB.

Risposta effettiva: Uno schema è una specifica che descrive la struttura di un oggetto, come una tabella. I data modelers devono definire delle tabelle in un database relazionale prima che gli sviluppatori possano eseguire il codice di aggiunta, rimozione e aggiornamento delle tabelle. Mentre nei document DB ciò non è necessario, gli sviluppatori possono creare delle collezioni e i documenti nelle collezioni semplicemente inserendoli nel database. Un vantaggio è il fatto che hanno una alta flessibilità nella gestione della struttura degli oggetti da memorizzare. Uno svantaggio è che il DBMS potrebbe non riuscire ad applicare regole basate sulla struttura dei dati (esempio: tutti i campi necessari sono presenti?)



9. Contesti in cui è meglio usare un documentDB.

Risposta:

Contesto: Un document DB è un database non relazionale che salva dati sotto forma di documenti, di solito XML o JSON. Permettono una grande flessibilità e garantiscono la gestione di strutture dati complesse, oltre a adottare un ridotto numero di collezioni per modellare i dati. Questo tipo di database sono schema-less: non devono essere definiti elementi documento “predefiniti”. Permettono anche operazioni complesse come le query e il filtering. I document database non prevedono operazioni di join fra le entità ma permettono i documenti innestati per rappresentare le relazioni fra le entità

Risposta effettiva: Dove le caratteristiche sopra sono necessarie e/o utili.

10. Come decidiamo se due documenti differenti possono andare nella stessa collezione?

Risposta:

Contesto: siamo in un document DB dove i documenti JSON sono gli elementi presenti nelle collezioni e sono il modo di salvare i dati.

Risposta effettiva: Le collezioni sono set di documenti, una collezione può salvare documenti di differenti tipi (nessuna specifica struttura o schema per un documento). In generale le collezioni dovrebbero salvare i documenti di uno stesso tipo di entità o che condividono uno scopo comune, ovvero vengono usati spesso insieme o in modo simile. Esempi di 2 entità:

```
{ "id" : 12334578,  
  "datetime" : "201409182210",  
  "session_num" : 987943,  
  "client_IP_addr" : "192.168.10.10",  
  "user_agent" : "Mozilla / 5.0",  
  "referring_page" : "http://www.example.com/page1"  
}
```

web clickstream data

```
{ "id" : 31244578,  
  "datetime" : "201409172140",  
  "event_type" : "add_user",  
  "server_IP_addr" : "192.168.11.11",  
  "descr" : "User jones added with sudo privileges"  
}
```

server log data

Metto le due istanze nella stessa entità chiamata “system event”:

Entity Name: **System Event**

```
{ "id" : 12334578,
  "datetime" : "201409182210",
  "doc_type": "click_stream",
  "session_num" : 987943,
  "client_IP_addr" : "192.168.10.10",
  "user_agent" : "Mozilla / 5.0",
  "referring_page" : "http://www.example.com/page1"
}

{ "id" : 31244578,
  "datetime" : "201409172140"
  "doc_type" : "server_log"
  "event_type" : "add_user"
  "server_IP_addr" : "192.168.11.11"
  "descr" : "User jones added with sudo privileges"
}
```

Mescolare tipi di documenti nella stessa raccolta può portare a più tipi di documenti in un blocco dati del disco. Questo può portare a inefficienze ogni volta che i dati vengono letti dal disco ma non utilizzati dall'applicazione che magari filtra i documenti in base al tipo. Se in una collezione grande ci sono molti tipi può convenire scannerizzare tutto il documento anziché usare un indice. Bisogna salvare tipi differenti di documenti nella stessa collezione solo se saranno frequentemente usati insieme nell'applicazione oppure se hanno un basso numero di differenze.

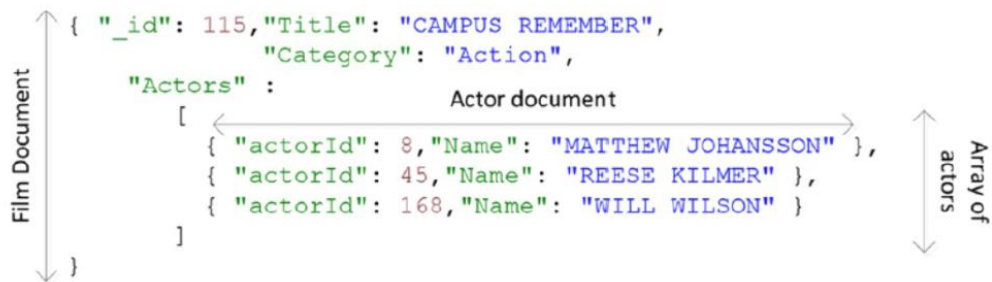
11. Differenza tra Document embedding e document linking?

Risposta

Contesto: Si utilizzano nei document database.

Risposta effettiva: Per il Data Modeling si possono usare due approcci differenti:

- **document embedding:** L'immagine di sotto è una soluzione che permette all'utente di ottenere un film e tutti i suoi attori in una singola operazione.



Il problema è che gli attori risultano duplicati nei vari documenti. In design complessi questo potrebbe portare a problemi e ad eventuali inconsistenze se qualche attributo dell'attore cambiasse. Inoltre, alcuni JSON DB hanno delle limitazioni per quanto riguarda la dimensione massima di un singolo documento.

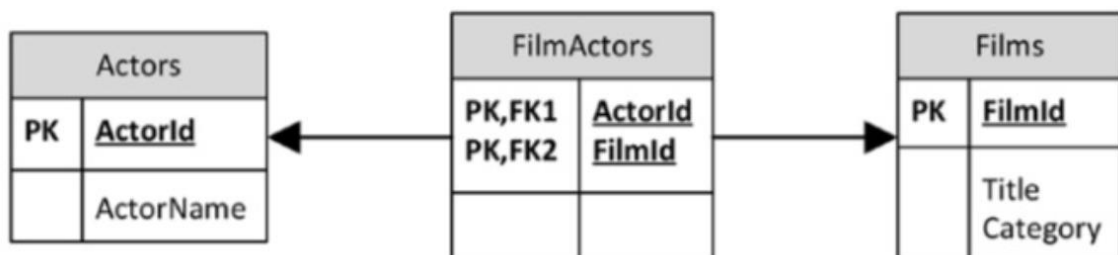
- **document linking:** Il document linking invece può essere spiegato con il seguente esempio:



Nella soluzione in figura, l'array di ID degli attori è stato integrato nel documento film. Gli ID possono essere usati per ottenere i documenti degli attori (in un'altra collezione) che appaiono in quel film. In questo caso almeno 2 collezioni di documenti devono essere definite. Tutto ciò risulta nel "tornare indietro" alla terza forma normale:



We are **rolling back** to a the third normal form!!!!



In generale il document linking è qualcosa di innaturale nei document database ma per alcuni carichi può fornire il migliore bilanciamento tra performance e manutenibilità.

Ricordiamo che non esiste un preciso equivalente della terza forma normale nei document DB che definisce un modello corretto.

Come si sceglie fra i due? Dipende dalla natura delle query da eseguire.

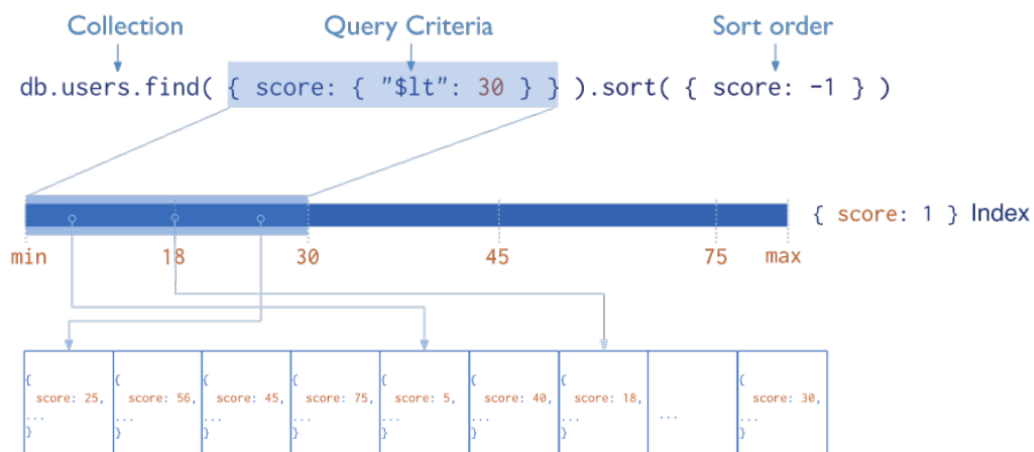
Vediamo nei due tipi di relazioni come compaiono queste due metodologie.

12. Parlami degli indici in MongoDB. A che livello sono definiti? Cosa sono i Compound Indexes?

Risposta:

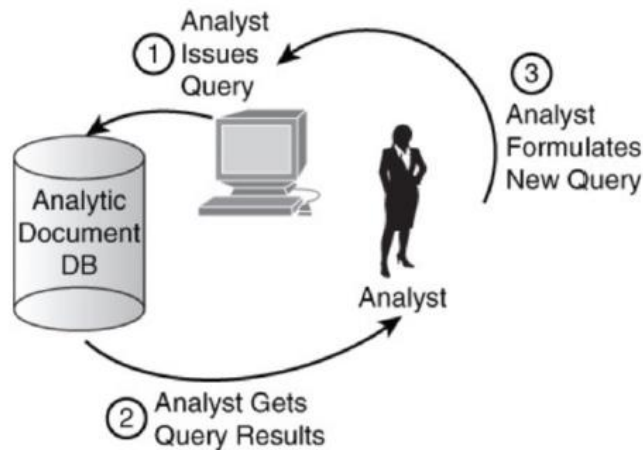
Contesto: Gli indici permettono di ottenere query più veloci e possono includere sia campi semplici che campi appartenenti a documenti o array innestati (embedded). Per evitare l'intera scansione del database complessivo, i DBMS per i document DB (ad esempio MongoDB) consentono la definizione di indici. Gli indici, come negli indici dei libri, sono un set strutturato di informazioni che collegano un attributo alle informazioni correlate ad esso (ex: Mario Rossi, ID = 4,5,6). In generale, gli indici sono strutture dati speciali che memorizzano una piccola porzione del set di dati della raccolta in un formato facile da attraversare. L'indice memorizza il valore di un campo specifico o di un set di campi, ordinato in base al valore del campo. L'ordinamento delle voci dell'indice supporta corrispondenze di uguaglianza efficienti e operazioni di query basate su intervalli. Spesso un indice è rappresentato come una lista di puntatori a documenti.

An Example of Index



I documenti nella collezione (raffigurati sotto l'indice) contengono dati completi, ma l'indice collega solo i campi essenziali (come score) a questi documenti.

Nella figura di seguito è presente uno schema classico di una read-heavy application (business intelligence e analytics applications):



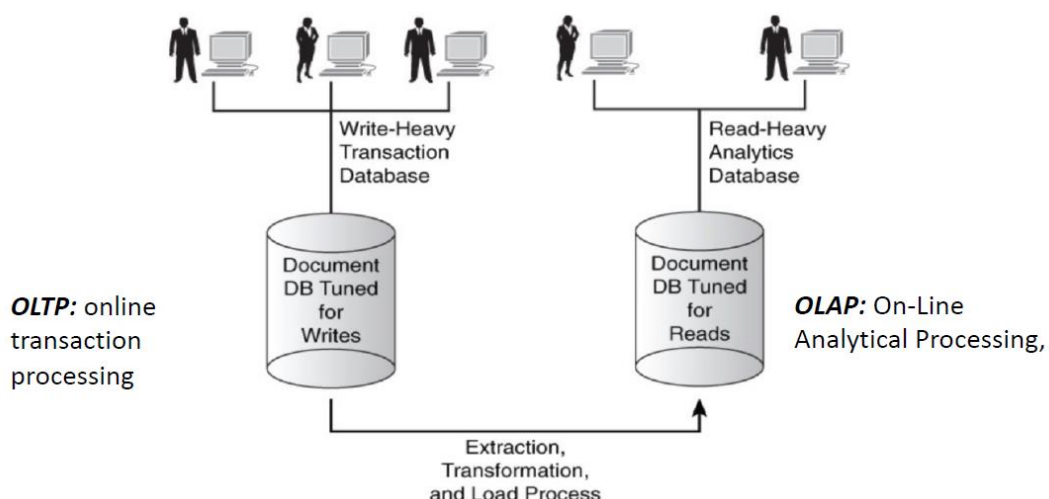
In questo tipo di applicazioni, l'uso di più indici consente all'utente di accedere rapidamente al database. Ad esempio, gli indici possono essere definiti per recuperare facilmente documenti che descrivono oggetti correlati a una specifica regione geografica o ad un tipo specifico.

Sono presenti anche le write-heavy application, come quella della trasmissione di informazioni sui camion (ogni tre minuti). Maggiore è il numero di indici adottati, maggiore è la quantità di tempo richiesto per chiudere un'operazione di scrittura poiché tutti gli indici devono essere aggiornati (e creati all'inizio). Riducendo il numero di indici, ci consente di ottenere sistemi con risposte rapide alle operazioni di scrittura. D'altro canto, dobbiamo accettare di gestire operazioni di lettura lente.

In conclusione, il numero e il tipo di indici da adottare devono puntare ad un compromesso.

Transactions Processing Systems

These systems are designed for fast write operation and targeted reads, as shown in the figure below:



Risposta effettiva:

Gli indici permettono l'esecuzione efficiente delle query in MongoDB. Senza indici, MongoDB deve scansionare ogni documento in una collezione, per selezionare i documenti che corrispondono alla query. Se esiste un indice appropriato per una query, MongoDB può

utilizzare l'indice per limitare il numero di documenti che deve ispezionare. Inoltre, MongoDB può restituire risultati ordinati utilizzando l'ordinamento nell'indice. MongoDB definisce gli indici a livello di collezione e supporta gli indici su qualsiasi campo o sottocampo dei documenti in una collezione.

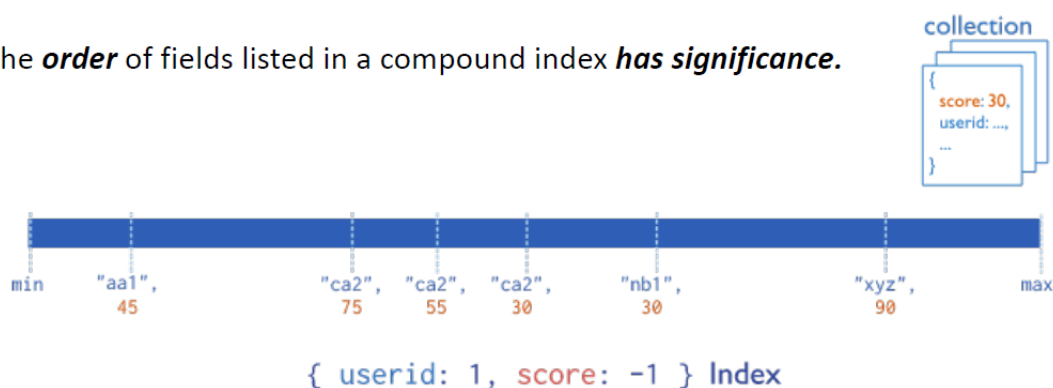
MongoDB crea un indice univoco sul campo `_id` durante la creazione di una collezione. Un indice univoco assicura che i campi indicizzati non memorizzino valori duplicati. L'indice `_id` impedisce ai client di inserire due documenti con lo stesso valore per il campo `_id`.

L'indice sul campo `_id` non può essere eliminato.

Di seguito un esempio di compound index:

In the following image we can see an example of user-defined index *on multiple fields*.

The **order** of fields listed in a compound index *has significance*.



Possiamo specificare una query di ordinamento su tutte le chiavi dell'indice o su un sottoinsieme. Tuttavia, le chiavi di ordinamento devono essere elencate nello stesso ordine in cui appaiono nell'indice. Ad esempio, un modello di chiave di indice { a: 1, b: 1 } può supportare un ordinamento su { a: 1, b: 1 } ma non su { b: 1, a: 1 }. Affinché una query utilizzi un indice composto per un ordinamento, la direzione di ordinamento specificata per tutte le chiavi deve corrispondere al modello di chiave di indice o corrispondere all'inverso del modello di chiave di indice. Ad esempio, un modello di chiave di indice { a: 1, b: -1 } può supportare un ordinamento su { a: 1, b: -1 } e { a: -1, b: 1 } ma non su { a: -1, b: -1 } o { a: 1, b: 1 }.

13. Quale architettura è meglio usare per programmare ad oggetti?

Risposta:

Contesto:

Risposta effettiva: è schemaless, viene violata l'encapsulation, il migliore è il document DB.

14. Definizione e differenza tra replicazione e sharding, specificare a cosa servono dal punto di vista delle web scale application.

Risposta:

Contesto: I più recenti NoSQL DBMS sono utilizzati in sistemi distribuiti, ovvero su molteplici server. I pro di questo approccio:

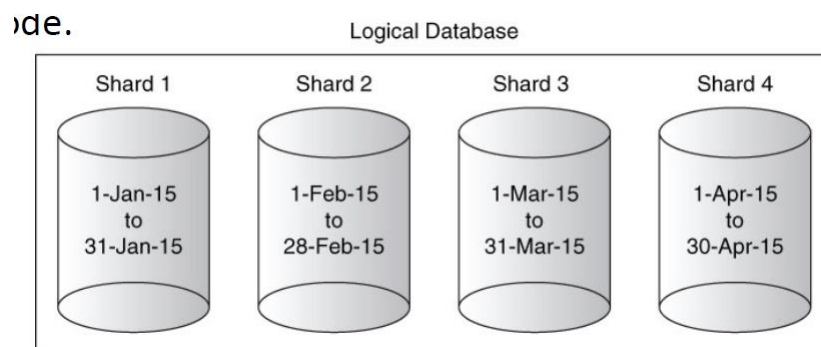
- Assicurano scalabilità, flessibilità, disponibilità
- Facile aggiungere e rimuovere nodi (grazie alla scalabilità orizzontale) invece che quella verticale
- Aggiungono strategie di fault tolerance

Un contro è il bilanciamento tra consistency e availability: per consistency si intende che tutti i nodi di un sistema vedono gli stessi dati contemporaneamente, se una modifica viene apportata a un dato, tutti i nodi vedono immediatamente il cambiamento attraverso invio di aggiornamenti. Mentre con disponibilità si intende che i dati devono essere sempre accessibili, dunque bisogna trovare una soluzione ai fallimenti, per esempio avere dei server di backup che intervengono quando uno si rompe. Ovviamente avere server di backup indica che devono essere aggiornati, quindi qui nasce il conflitto tra le due proprietà.

Risposta effettiva:

Lo sharding o partizionamento orizzontale è il processo di divisione dei dati in blocchi o chunks.

- Ogni blocco, etichettato come shard, viene distribuito su un nodo specifico (server) di un cluster.
- Ogni nodo può contenere solo uno shard.
- In caso di replica dei dati, uno shard può essere ospitato da più di un nodo.



Vantaggi dello sharding:

- Consente di gestire carichi pesanti e l'aumento degli utenti del sistema (load balancing)

- I dati possono essere facilmente distribuiti su un numero variabile di server che possono essere aggiunti o rimossi su richiesta (scalabilità orizzontale)
- Più economico del vertical scaling (aggiunta di RAM e dischi, aggiornamento delle CPU a un singolo server...)
- In combinazione con la replication, garantisce una alta availability del sistema e risposte rapide.

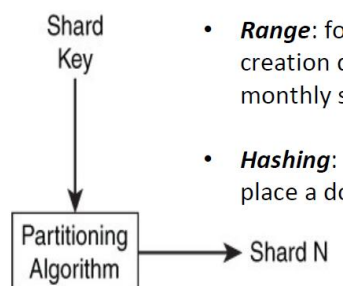
Per implementare lo sharding, i progettisti di document databases devono selezionare una chiave di shard e un metodo di partizionamento. Una chiave di shard è uno o più campi, che esistono in tutti i documenti in una raccolta, che viene utilizzata per separare i documenti.

Esempi di chiavi di shard possono essere: ID documento univoco, Nome, Data, data di creazione, Categoria o tipo, Regione geografica...

In realtà, qualsiasi campo atomico in un documento può essere scelto come chiave di shard.

Esistono tre categorie principali di algoritmi di partizione, in base a:

- Intervallo: ad esempio, se tutti i documenti in una raccolta avessero un campo data di creazione, potrebbe essere utilizzato per suddividere i documenti in frammenti mensili.
- Hashing: una funzione hash può essere utilizzata per determinare dove posizionare un documento. Può essere utilizzato anche un hashing coerente.
- Elenco: ad esempio, immaginiamo un database di prodotti con diversi tipi (elettronica, elettrodomestici, articoli per la casa, libri e vestiti). Questi tipi di prodotto potrebbero essere utilizzati come chiave di frammento per allocare i documenti su cinque server diversi.



La replicazione garantisce una alta availability e consiste nel salvataggio di più copie dei dati nei nodi di un cluster. Il numero di repliche di dati è spesso un parametro da impostare. Maggiore è il numero di repliche, minore è la probabilità di perdere dati, minore è il tempo di risposta. Minore è il numero di repliche invece, migliore è la prestazione dei sistemi, ma maggiore è la probabilità di perdere dati.

Un basso numero di repliche può essere utilizzato ogni volta che i dati vengono facilmente rigenerati e ricaricati.

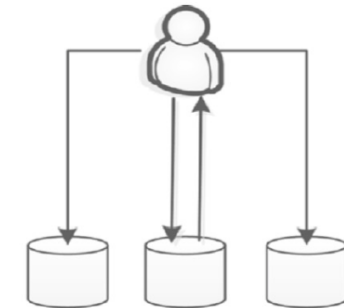
In sintesi le differenze sono:

La replicazione favorisce l'availability, lo sharding il load balancing e la scalabilità orizzontale.

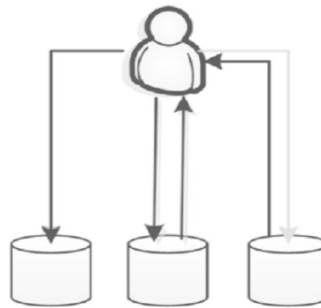
(Da qui in poi credo non serva nella risposta sennò viene troppo lunga)

Di seguito esempi sulla replicazione:

Write/Read operations with Replicas



N=3 W=3 R=1
Slow writes, fast reads, consistent
There will be 3 copies of the data.
A write request only returns when all 3 have written to disk.
A read request only needs to read one version.



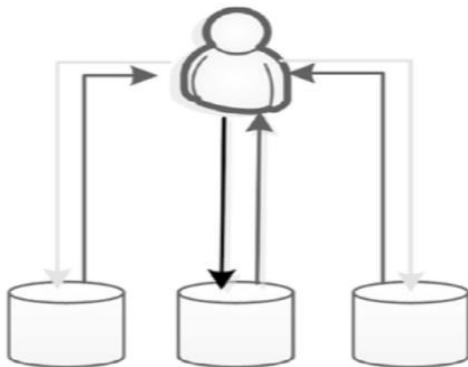
N=3 W=2 R=2
Faster writes, still consistent (quorum assembly)
There will be 3 copies of the data.
A write request returns when 2 copies are written – the other can happen later.
A read request reads 2 copies make sure it has the latest version.

N = # of replicas

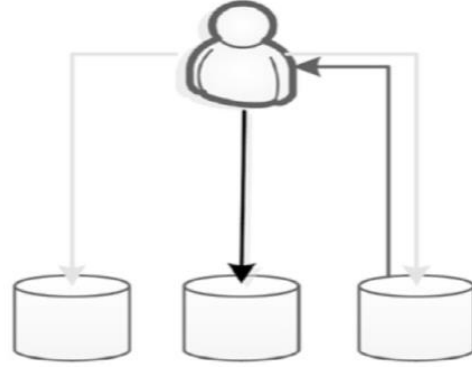
W = # of copies to be written before the write can complete

R = # of copies to be read for reading a data record

Write/Read operations with Replicas



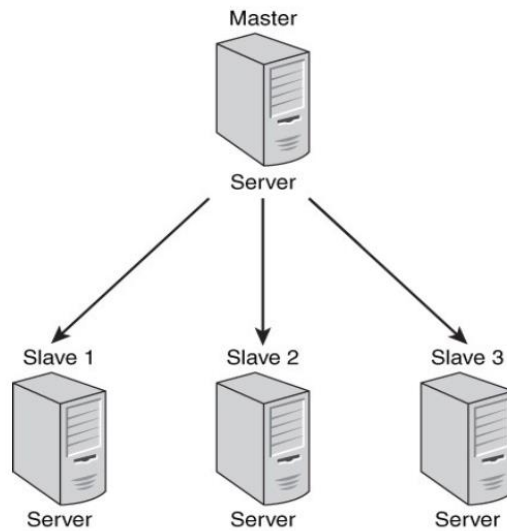
N=3 W=1 R=N
Fastest write, slow but consistent reads
There will be 3 copies of the data.
A write request returns once the first copy is written – the other 2 can happen later.
A read request reads all copies to make sure it gets the latest version.
Data might be lost if a node fails before the second write.



N=3 W=1 R=1
Fast, but not consistent
There will be 3 copies of the data.
A write request returns once the first copy is written – the other 2 can happen later.
A read request reads a single version only: it might not get the latest copy.
Data might be lost if a node fails before the second write.

La replicazione può essere master-slave o masterless.

Master e slave:



Il master è un server che accetta richieste di scrittura e lettura ed è responsabile della raccolta di tutte le modifiche dei dati e dell'aggiornamento dei dati per tutti gli slave. Gli slave possono rispondere solo alle richieste di lettura

Questa architettura è particolarmente adatta per applicazioni con una crescente richiesta di operazioni di lettura.

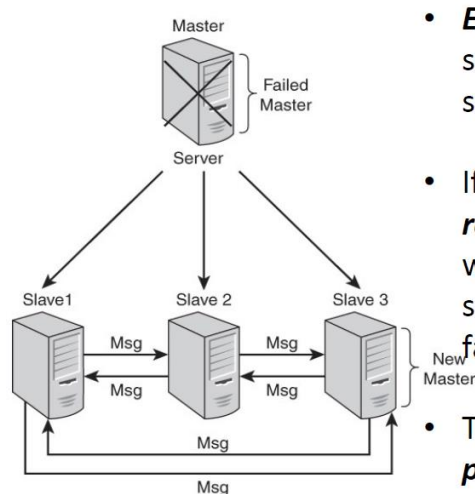
Pro:

- Architettura molto semplice: solo il master comunica con altri server.
- Non c'è bisogno di coordinare le operazioni di scrittura o risolvere conflitti tra più server che accettano scritture.

Contro:

- se il master fallisce, il cluster non può accettare scritture.
- In questo caso, l'intero sistema fallisce o almeno perde una capacità critica, come accettare scritture.

Maggiore complessità porta ad avere migliore availability:



- **E**
S
S
- **I**
r
v
S
f
i
T
p

- Ogni singolo server slave può inviare un semplice messaggio per chiedere a un server casuale nel cluster se è ancora attivo.

- Se un certo numero di server slave non riceve un messaggio dal master entro un certo periodo di tempo, gli slave possono determinare che il master ha fallito.
- Gli slave avviano un protocollo per promuovere uno degli slave come nuovo master.

Un caso d'uso per la masterless è il seguente: applicazioni in cui il numero di scritture può aumentare e raggiungere picchi! Esempio: il sito web che vende biglietti per concerti. L'architettura di replica master-slave subirà un numero crescente di richieste di scrittura (solo il master accetta le scritture e tutti gli aggiornamenti devono essere inviati agli slave).

Soluzione: un modello di replica in un cluster di server in cui tutti i nodi possono accettare richieste di scrittura.

15. Spiegare in cosa consistono replicazione e sharding e a quali requisiti non funzionali portano vantaggi

Risposta:

Stessa di prima

16. Come sharding e partitioning contribuiscono ad avere la availability e in cosa differiscono? Definire le due proprietà.

Risposta:

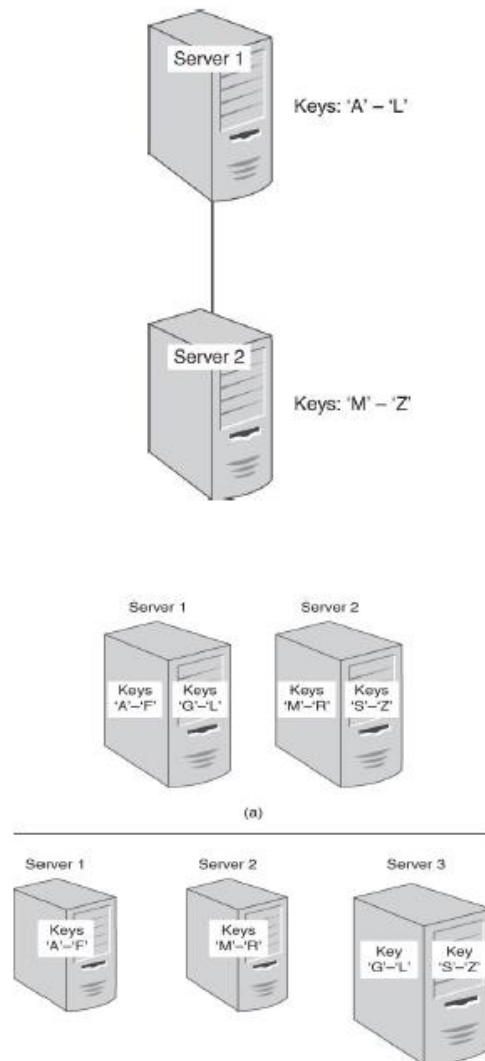
Contesto: I più recenti NoSQL DBMS sono utilizzati in sistemi distribuiti, ovvero su molteplici server. I pro di questo approccio:

- Assicurano scalabilità, flessibilità, disponibilità
- Facile aggiungere e rimuovere nodi (grazie alla scalabilità orizzontale) invece che quella verticale
- Aggiungono strategie di fault tolerance

Contro:

- Bilanciare la consistenza con la disponibilità
- Proteggere dai fallimenti della rete che possono lasciare nodi isolati

Nell'immagine possiamo vedere un esempio generale di Data partitioning:

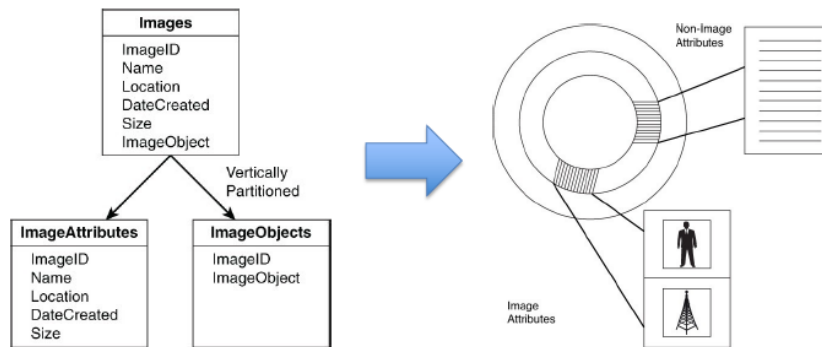


Il partitioning si riferisce al processo di suddivisione di un singolo database in parti distinte e non sovrapposte, chiamate partizioni. Le partizioni di un insieme di dati (chiamate **shard** o **chunk**) possono essere gestite da uno o più nodi diversi di un sistema distribuito. L'obiettivo principale è quello di **bilanciare (load balancing)** uniformemente i carichi di scrittura e lettura tra i server del sistema. Inoltre, deve essere facile ribilanciare i dati in seguito all'aggiunta o alla rimozione di un nodo nel cluster.

Si definisce **Partition Key** la chiave che identifica una specifica partizione nel quale troveremo il valore di interesse.

Lo svantaggio di questo metodo è che aggiornare le repliche richiede tempo ed energie computazionali.

Il primo metodo di partitioning è il partizionamento verticale: è una tecnica per migliorare le prestazioni del database (relazionale) separando le colonne di una tabella relazionale in più tabelle separate.



Lo **sharding** o **partizionamento orizzontale** è la **seconda tecnica di partitioning**, è il processo di suddivisione dei dati in blocchi. Ogni **shard** (blocco) è suddiviso in uno specifico **nodo** (server) di un **cluster**. Ogni nodo può contenere un solo shard e, in caso di replica dei dati, uno shard può essere ospitato in più nodi.

I vantaggi di effettuare lo sharding sono che:

- consente di gestire carichi pesanti all'aumentare degli utenti del sistema.
- i dati possono essere facilmente distribuiti su un numero variabile di server che, a seconda delle esigenze, possono essere aggiunti o rimossi.
- è più economico del ridimensionamento verticale.
- se viene utilizzato in combinazione con le repliche, garantisce l'availability del sistema e una risposta rapida.

Per Availability (Disponibilità) ci si riferisce alla capacità del sistema di rimanere operativo e accessibile agli utenti anche in caso di guasti. Una strategia di partitioning che distribuisce i dati su più nodi contribuisce a migliorare la disponibilità, poiché un problema su un nodo non blocca l'intero sistema.

Risposta effettiva:

Il **vertical partitioning** non aumenta la disponibilità allo stesso modo dello sharding, ma migliora le prestazioni del singolo database permettendo di accedere solo alle colonne rilevanti per una determinata query. Tuttavia, se il nodo o il server che contiene una partizione fallisce, potrebbe comunque compromettere l'accesso ai dati di quella tabella.

Lo **sharding** suddivide i dati per righe, distribuendo ogni blocco (shard) su nodi separati. Ad esempio, in un sistema con due server, le chiavi da 'A' a 'L' potrebbero essere su un server e le chiavi da 'M' a 'Z' su un altro. In questo modo, ogni shard rappresenta un subset indipendente di dati. Lo sharding offre **alta disponibilità**, poiché ogni shard risiede su un nodo separato: se uno shard fallisce, solo una parte dei dati diventa inaccessibile, mentre gli altri shard continuano a funzionare. Inoltre, è possibile replicare gli shard su più nodi per garantire la continuità del servizio in caso di guasto.

Modeling hierarchies

17. Definizione di Keyspace, Row keys, Column family e Column nei Columnar Database

Risposta:

Contesto: Quando elaboriamo dati per fare analisi, nella maggior parte dei casi, non siamo interessati a recuperare tutte le informazioni di ogni singolo record. In effetti, siamo interessati spesso, ad esempio, a recuperare tutti i valori di un attributo di un set di record (per fare grafici di tendenza o calcolare statistiche, ad esempio la media di un attributo). Quando si ha a che fare con l'archiviazione di record basata su righe, dobbiamo accedere a tutti i record del set considerato per recuperare solo i valori di un attributo. Se tutti i valori di un attributo sono raggruppati insieme sul disco (o nel blocco di un disco), l'attività di recupero dei valori di un attributo sarà più veloce di un'archiviazione di record basata su righe.

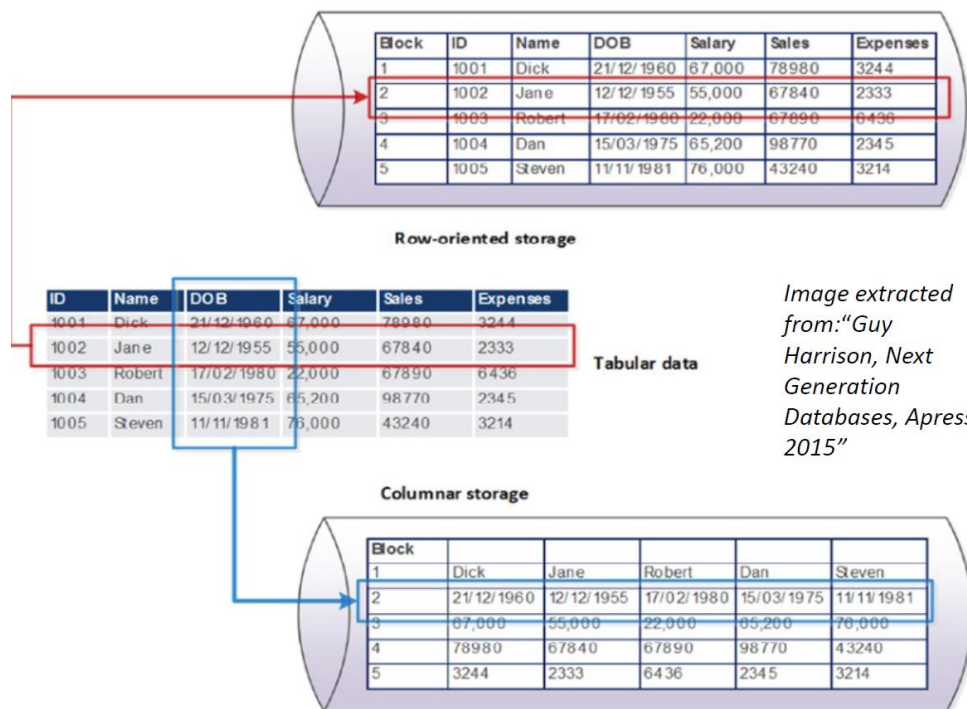
Row Storage

Last Name	First Name	E-mail	Phone #	Street Address

Columnar Storage

Last Name	First Name	E-mail	Phone #	Street Address

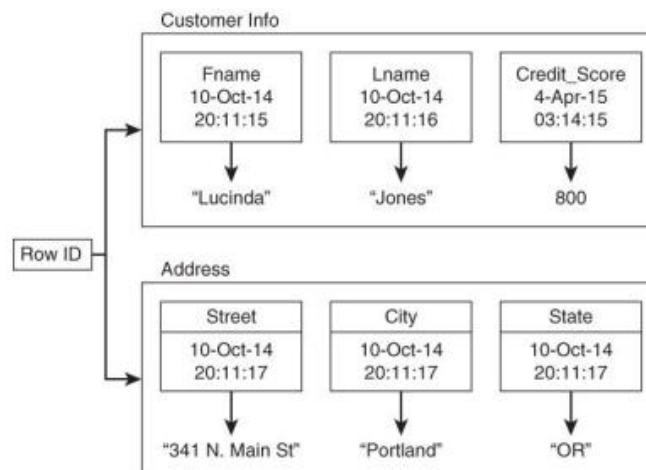
(la parte seguente del contesto può essere omessa per evitare di rendere troppo lunga la risposta)



Se i dati sono salvati per righe e vogliamo per esempio fare una media di un attributo (abbiamo 5 righe come in figura), devo fare lo scan di 5 blocchi. Nel caso di una memorizzazione a colonna, basta accedere ad un solo blocco. In generale se abbiamo query su molteplici righe scegliere un columnar DB è una ottima scelta.

I columnar DB hanno basse performance durante le modifiche di una singola riga. L'overhead di leggere le righe può essere parzialmente ridotto con il caching e le proiezioni multicolonna (salvare molteplici colonne insieme in un disco). In generale, gestire una riga significa accedere a più di un blocco nel disco.

Un esempio di columnar DB importante è il database NoSQL "Big Table" di Google. È basato sulle Column con un controllo dinamico su di esse, i valori sono indicizzati da un identificatore di row, name of column e timestamp. Le scritture e letture sono atomiche. Le row sono mantenute in modo ordinato.



Ogni record di dati è una row composta da molte column families: in figura il *Row ID* indica sia la column family *Customer info* che quella *Address*, ma soprattutto definisce anche la riga di valori delle colonne.

Ogni column family consiste in un set di related column (spesso usate insieme): *Street, City, State* nella column family *Address* in figura. I valori delle colonne dentro una column family sono tenuti insieme nel disco, mentre i valori delle stesse righe ma appartenenti a famiglie diverse possono essere salvati lontano.

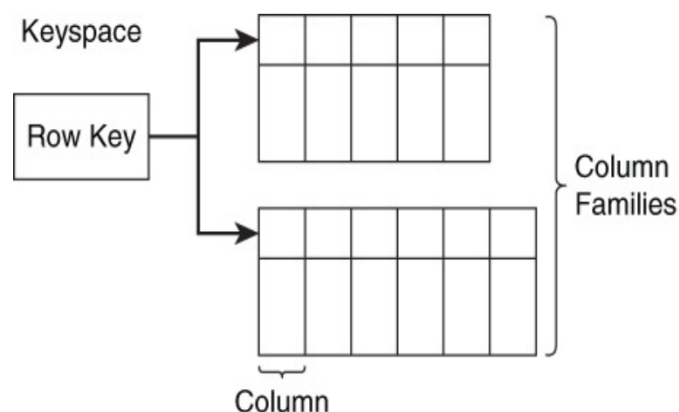
Le applicazioni possono aggiungere o rimuovere le colonne dentro alle famiglie.

Un valore è indicizzato da:

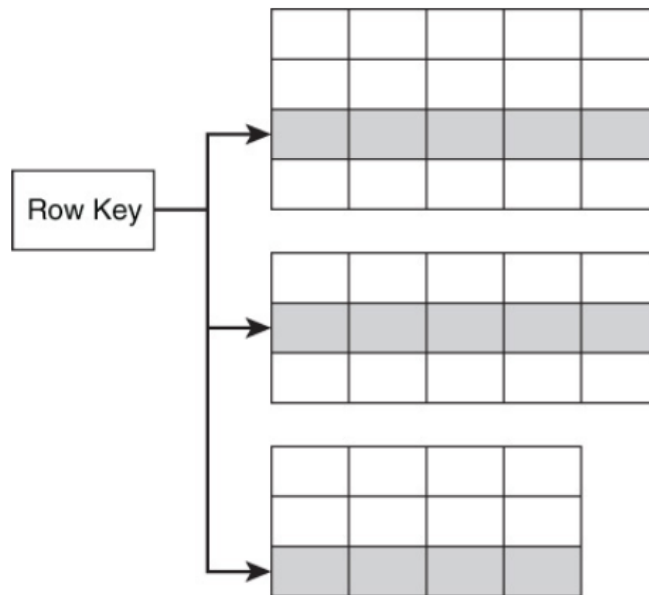
- row identifier (simile al ruolo che ha una chiave primaria nel database relazionale)
- column name, che identifica la colonna (simile alla chiave nei key value DB)
- timestamp che permette l'esistenza di molteplici versioni di uno stesso valore di una colonna

Risposta effettiva:

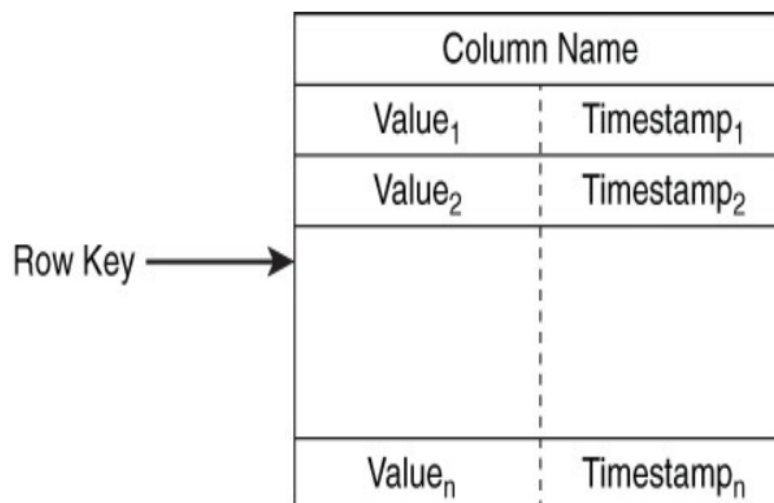
- Il Keyspace è il top-level logical container. Esso salva le Column families, le Row keys e le relative strutture dati. Tipicamente c'è un solo keyspace per applicazione.



- Le Row key sono una delle componenti usate per identificare unicamente i valori salvati in un DB ma sono anche usate per partizionare e per ordinare i dati. Gli algoritmi per il partizionamento e l'ordinamento dipendono dagli specifici database. Ad esempio, Cassandra adotta il *partitioner*, ovvero un oggetto specifico che viene utilizzato per ordinare i dati e per distribuire randomicamente le righe nei nodi.



- Le Column, insieme alle Row key e ad un Version stamp, identificano unicamente i valori. I valori possono essere tipizzati o meno. I timestamp o i Version stamp sono un modo per ordinare i valori di una Column. Le Column in una Column family possono essere modificate dinamicamente



- Le Column che sono usate frequentemente insieme sono raggruppate nella stessa Column family. Le column family sono analoghe alle tabelle nei database relazionali e devono essere definite a priori. Le righe nelle Column family sono ordinate e “versionate”

In generale, nei Columnar database, le modifiche ai dati, come aggiornamenti o inserimenti, non vengono necessariamente applicate direttamente al dato esistente nel database in tempo reale. Piuttosto, si utilizza un sistema di *references* o puntatori alle modifiche. Questo significa che ci sono modifiche basate su timestamp: ogni modifica è registrata come un nuovo record con un timestamp, e i dati più recenti vengono selezionati quando si interroga il database.

Street	City	State	Province	Zip	Postal Code	Country
178 Main St.	Boise	ID		83701		U.S.
89 Woodridge	Baltimore	MD		21218		U.S.
293 Archer St.	Ottawa		ON		K1A 2C5	Canada
8713 Alberta DR	Vancouver		BC		V5K 0A1	Canada

18. Bloom Filters

Risposta:

Contesto: I column DB più recenti sono progettati per garantire elevata availability e scalability, insieme a diversi livelli di consistency. Cluster e partizioni sono fondamentali per l'implementazione distribuita dei database. Una partizione è un sottoinsieme logico di un database. Le partizioni sono solitamente utilizzate per archiviare un set di dati in base ad alcuni attributi dei dati nei database a colonne, possono essere generate sulla base di:

- Un intervallo di valori, come il valore di un ID di riga/nome di colonna
- Un valore hash, come il valore hash di un nome di colonna
- Un elenco di valori, come i nomi di stati o province
- Un composto di due o più delle opzioni di cui sopra

Risposta effettiva: I bloom filters sono una struttura dati probabilistica. Aiutano a ridurre il numero di operazioni di lettura, evitando la lettura nelle partizioni dove sicuramente non è presente il dato cercato. Un bloom filter testa se un elemento è parte di un set o meno: la risposta è “no” o “forse”. Dunque, i falsi positivi possono essere ottenuti. Se la risposta è no sicuramente il dato non è presente in quella partizione.

Member Function			Bloom Filter		
Input Set	Test Element	In Set	Input Set	Test Element	In Set
{a,b,c}	a	Yes	{a,b,c}	a	Yes
{a,b,c}	c	Yes	{a,b,c}	c	Yes
{a,b,c}	e	No	{a,b,c}	e	Yes
			{a,b,c}	f	No
			{a,b,c}	g	No



 Low Probability but Possible

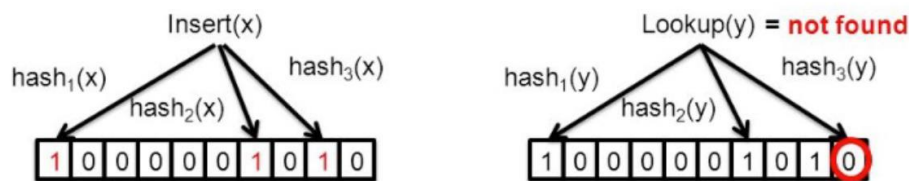
image extracted from: "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, '015

Consideriamo una specifica partizione. Un bloom filter per questa partizione può essere un array di n valori binari. All'inizio, tutti gli elementi sono settati a 0. Consideriamo anche un set di $k \ll n$ funzioni hash che mappano ogni dato (che deve essere salvato in una partizione) dentro un elemento di un array. Quando un nuovo dato è aggiunto (scrittura) alla partizione, tutte le funzioni hash sono calcolate e i corrispondenti bit sono settati a 1. Quando

una lettura è richiesta, tutte le funzioni hash sono calcolate e l'accesso alla partizione è fatto se e solo se tutti i corrispettivi elementi dell'array sono settati a 1. Il numero di falsi positivi dipende da n e k .

Esempio:

- Let consider a Bloom Filter with $n = 10$ and $k = 3$.
- The **Insert** function describes a **write** operation
- The **Lookup** function describes a **read** operation



Controllare che un elemento è presente o aggiungerne uno richiede un tempo costante. La probabilità che ci siano falsi positivi decresce se n cresce, mentre cresce se il numero di elementi salvati nel bloom filter cresce. È impossibile rimuovere un elemento da un semplice Bloom Filter, perché consentire la rimozione porterebbe a falsi negativi, che non sono ammessi. Un falso negativo significherebbe che il filtro segnala erroneamente che un elemento non è presente quando invece lo è.

19. Anti-entropy process

Risposta:

Contesto: Il livello di consistency si riferisce alla consistency tra copie di dati su diverse repliche, ciò dipende dai requisiti specifici dell'applicazione.

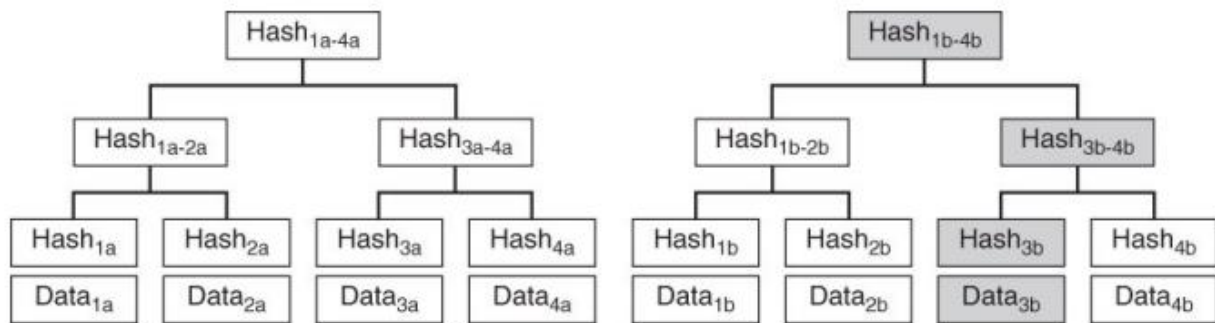
Possiamo considerare diversi livelli di Consistency:

- Strict Consistency: tutte le repliche devono avere gli stessi dati
- Low Consistency: i dati devono essere scritti in almeno una replica
- Moderate Consistency: tutte le soluzioni intermedie

Il livello di consistency è strettamente correlato al processo di replicazione. Il processo di replicazione riguarda dove posizionare le repliche e come mantenerle aggiornate.

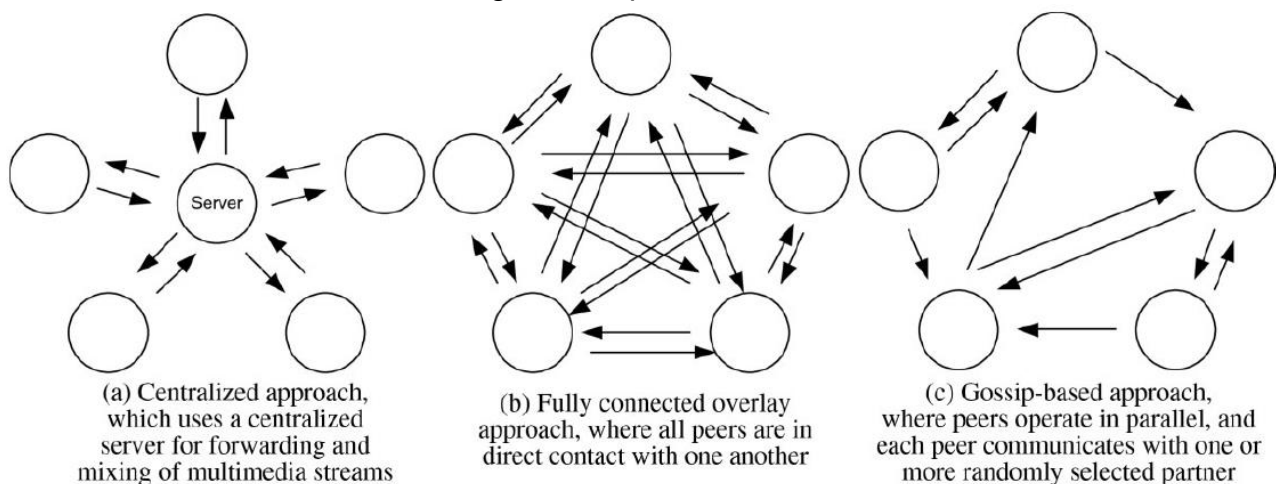
- Di solito, c'è un server predefinito per essere la prima replica che è responsabile di identificare la posizione delle altre repliche tra i server disponibili.
- Il server primario è spesso determinato da una funzione hash.
- Ogni database ha le sue strategie di replicazione.

Risposta effettiva: L'anti-entropia è il processo di rilevamento delle differenze nelle repliche. È importante rilevare e risolvere le incongruenze tra le repliche scambiando una piccola quantità di dati. Gli alberi hash possono essere scambiati tra server per controllare facilmente le differenze perché confrontare un valore hash è molto più veloce che scansionare l'intera partizione. Ogni foglia dell'albero contiene il valore hash di un set di dati nella partizione. Ogni nodo padre contiene il valore hash degli hash dei suoi nodi figli.



1. Un server riceve periodicamente l'albero degli altri nodi.
2. Il server lo confronta con il suo albero.
3. Se le due radici sono uguali: le copie sono identiche e l'algoritmo si ferma.
4. Altrimenti, navigherà nell'albero e confronterà il figlio della radice.
5. Seguendo il nodo diverso, verrà individuata la partizione diversa.
6. Il server con la partizione più vecchia viene aggiornato dall'altro.

Nei sistemi distribuiti, ogni nodo deve essere a conoscenza dello stato degli altri nodi. Nei piccoli cluster, ogni nodo può scambiare messaggi con ogni altro nodo. Il numero totale di messaggi da scambiare per una comunicazione completa è uguale a $n \times (n-1)/2$, dove n è il numero di nodi di un cluster. Di seguito vari protocolli di comunicazione:



OLTP OLAP

Delta store

projections

cluster e partitions

HBase

Cassandra

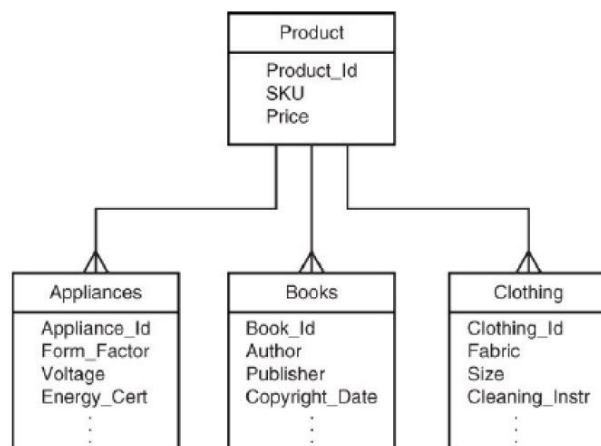
Consistency level

Replication

Esercizi pratici

1. Disegnare tabella vista a lezione dove c'è product, book clothing appliance, definire row key column family e column table, inserire due record con clothing id e uno con solo appliance ID (va disegnata la tabella)

Tabella:



In questo caso se utilizziamo un columnar DB si possono memorizzare tutti gli attributi presenti in figura in una sola riga, quindi ci saranno alcune righe che memorizzano più valori colonna. Possiamo definire nella figura seguente la struttura della column table:



Le column families sono: **Product_columns**, **Appliance_columns**, **Clothing_columns** e **Book_columns**. La column table include tutte e 4 le families. Ogni column family ha gli attributi presenti sopra nell'esempio relazionale.

Il row ID identifica una singola riga di questa tabella. Ogni riga memorizza solo le colonne rilevanti, ad esempio un prodotto di abbigliamento non avrà colonne relative agli elettrodomestici o ai libri. Solo le colonne rilevanti per una certa categoria di prodotto saranno valorizzate, mentre le altre resteranno vuote.

Row ID	Product_columns	Clothing_columns	Appliance_columns
Clothing-1	Product_ID: 101, SKU: C-001, Price: 50.00	Fabric: Cotton, Size: M, Cleaning_Instr: Hand Wash	-
Clothing-2	Product_ID: 102, SKU: C-002, Price: 70.00	Fabric: Wool, Size: L, Cleaning_Instr: Dry Clean	-
Appliance-1	Product_ID: 201, SKU: A-001, Price: 150.00	-	Voltage: 220V, Form_Factor: Compact, Energy_Cert: A+

- Ogni Row ID rappresenta un prodotto specifico.
- Ogni riga utilizza solo le colonne pertinenti per quel tipo di prodotto:
Clothing riempie solo le colonne di Clothing_columns.
Appliance riempie solo le colonne di Appliance_columns.
- Le famiglie di colonne vuote (es. Appliance_columns per Clothing) rimangono **non memorizzate**, grazie alla natura "sparse" del database.

2. Relazione one-to-many tra studenti ed esami, disegnare l'UML, definire uno o più requisiti funzionali per una soluzione document embedding e stesso per il document linking, scrivere in JSON i documenti per entrambi i casi

L'utente come campi ha nome, cognome e data di nascita, gli esami hanno la data, il voto e il nome

Esempio:

Student1:

Name: pippo

Surname: Pluto

DOB:

Exam1:

Date:

Mark:....

Subject:

Exam2:

Date:

Mark:....

Subject:

UML:



A. Document Embedding

Tutti i dati di uno studente e dei suoi esami sono memorizzati in un unico documento.

- **Requisiti Funzionali:**

1. Accesso rapido a tutti gli esami di uno studente.
2. Ideale per query che richiedono i dettagli dello studente con tutti i suoi esami.

B. Document Linking

Ogni studente ha un documento separato e ogni esame è memorizzato come un documento separato, collegato tramite una chiave.

- **Requisiti Funzionali:**

1. Migliore gestione di esami indipendenti dallo studente.
2. Permette aggiornamenti indipendenti tra studente ed esami.

Document embedding:

```
{
  "Student_ID": 1,
  "Name": "Pippo",
  "Surname": "Pluto",
  "DOB": "2000-01-15",
  "Exams": [
    {
      "Exam_ID": 101,
      "Date": "2024-11-01",
      "Mark": 28,
      "Subject": "Mathematics"
    }
  ],
}
```

```
{
  "Exam_ID": 102,
  "Date": "2024-11-15",
  "Mark": 30,
  "Subject": "Physics"
}
]
```

Document linking:

Studente:

```
{
  "Student_ID": 1,
  "Name": "Pippo",
  "Surname": "Pluto",
  "DOB": "2000-01-15"
}
```

Esame:

```
{
  "Exam_ID": 101,
  "Student_ID": 1,
  "Date": "2024-11-01",
  "Mark": 28,
  "Subject": "Mathematics"
}
```

Considerazioni finali:

Document Embedding:

- Pro: Ideale per query congiunte (es. tutti gli esami di uno studente).
- Contro: Poco flessibile per modifiche frequenti ai dati degli esami.

Document Linking:

- Pro: Consente aggiornamenti indipendenti e scalabilità per un gran numero di esami.
- Contro: Richiede una gestione delle relazioni più complessa.

3. Esercizio sui document DB: ti do entità e relazioni, fornisci una motivazione per usare il document embedding e una per il document linking, poi abbozzare i documenti nei due casi

Entità e Relazioni (Esempio)

Immaginiamo un'applicazione di gestione di un sistema di **e-commerce**. Le principali entità sono:

- **Prodotti** (Product)
 - nome
 - prezzo
 - categoria
 - descrizione
 - recensioni
- **Clienti** (Customer)
 - nome
 - email
 - indirizzo
 - ordini effettuati
- **Ordini** (Order)
 - data dell'ordine
 - stato dell'ordine
 - prodotti acquistati
 - cliente

Relazioni:

- Un **ordine** è associato a un solo **cliente**.
- Un **ordine** può contenere più **prodotti**.
- Un **cliente** può avere molti **ordini**.
- Un **prodotto** può avere molte **recensioni**.

Document Embedding

Il **document embedding** consiste nel "incorporare" entità e le relative informazioni in un singolo documento, come se fosse un "documento contenitore". È vantaggioso quando si prevede di leggere l'entità completa in una singola operazione di lettura. In generale, il document embedding è utile quando le entità hanno una relazione "forte" e si usano insieme frequentemente.

Motivazione per l'uso del document embedding: Nel caso di un **ordine**, potremmo voler raggruppare tutte le informazioni relative all'ordine in un unico documento. Ciò è particolarmente utile in un contesto in cui si desidera recuperare tutte le informazioni di un ordine in un solo colpo, ad esempio quando l'utente verifica lo stato del suo ordine, che include anche i dettagli dei prodotti acquistati e le informazioni sul cliente.

Documento con document embedding (Ordine):

```
{
  "_id": "order123",
  "data_ordine": "2024-11-19",
  "stato_ordine": "In elaborazione",
  "cliente": {
    "nome": "Mario Rossi",
    "email": "mario.rossi@email.com",
    "indirizzo": "Via Roma 123, Milano"
  },
  "prodotti": [
    {
      "id_prodotto": "prod001",
      "nome": "Smartphone XYZ",
      "prezzo": 499.99,
      "categoria": "Elettronica",
      "quantità": 1
    },
    {
      "id_prodotto": "prod002",
      "nome": "Cuffie Bluetooth",
      "prezzo": 89.99,
      "categoria": "Accessori",
      "quantità": 2
    }
  ]
}
```

Document Linking

Il **document linking** prevede l'uso di riferimenti tra i documenti, piuttosto che incapsulare tutte le informazioni in un singolo documento. Questo approccio è utile quando le entità sono relativamente indipendenti, ma è necessario stabilire connessioni tra di esse.

Motivazione per l'uso del document linking: Nel caso di un **cliente** e dei **prodotti**, possiamo separare queste entità in documenti distinti e utilizzare i riferimenti (ID) per collegarli. Questo è utile quando le entità cambiano frequentemente, ad esempio, un prodotto può essere aggiornato senza dover aggiornare ogni ordine in cui è stato acquistato.

Documento con document linking (Ordine):

```
{
  "_id": "order123",
  "data_ordine": "2024-11-19",
  "stato_ordine": "In elaborazione",
  "cliente_id": "customer123", // Riferimento al cliente
  "prodotti": [
    {
      "id_prodotto": "prod001", // Riferimento al prodotto
      "quantità": 1
    },
    {
      "id_prodotto": "prod002", // Riferimento al prodotto
      "quantità": 2
    }
  ]
}
```

Documento Cliente:

```
{
  "_id": "customer123",
  "nome": "Mario Rossi",
  "email": "mario.rossi@email.com",
  "indirizzo": "Via Roma 123, Milano"
}
```

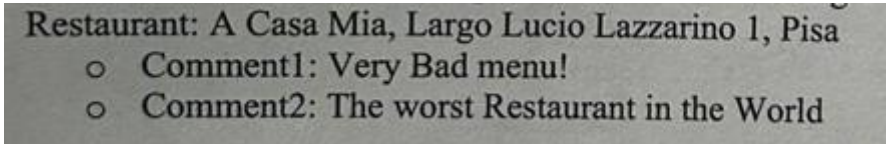
Documento Prodotto:

```
{  
  "_id": "prod001",  
  "nome": "Smartphone XYZ",  
  "prezzo": 499.99,  
  "categoria": "Elettronica"  
}
```

Conclusioni

- **Document Embedding:** Ideale quando si vuole raggruppare tutte le informazioni relative a una singola entità in un unico documento, riducendo la complessità nelle letture ma aumentando la ridondanza.
- **Document Linking:** Utile quando le entità sono relativamente indipendenti e si vogliono separare per facilitare aggiornamenti frequenti, riducendo la ridondanza ma aumentando la necessità di seguire i riferimenti tra documenti.

4. Supponiamo di considerare una relazione one-to-many tra l'entità ristorante e la entità recensioni tale che un ristorante abbia molte recensioni e una recensione sia associata ad un solo ristorante. Discuti con dettagli come questa relazione può essere implementata in un document DB, fornendo un esempio in formato JSON del DB per ognuna delle possibili implementazioni considerando i seguenti dati.



```
Restaurant: A Casa Mia, Largo Lucio Lazzarino 1, Pisa  
  o Comment1: Very Bad menu!  
  o Comment2: The worst Restaurant in the World
```

Risposta:

- Document embedding

In questo approccio, tutte le recensioni relative a un ristorante sono incorporate direttamente all'interno del documento che rappresenta il ristorante. È utile quando le recensioni sono strettamente legate al ristorante e devono essere recuperate insieme frequentemente.

```
{
```

```

"restaurant_id": "1",
"name": "A Casa Mia",
"address": "Largo Lucio Lazzarino 1, Pisa",
"reviews": [
  {
    "comment_id": "1",
    "comment": "Very Bad menu!"
  },
  {
    "comment_id": "2",
    "comment": "The worst Restaurant in the World"
  }
]
}

```

Vantaggi:

- Migliora le prestazioni quando si recuperano i dati del ristorante insieme alle recensioni.
- Meno operazioni di join, poiché tutti i dati sono nello stesso documento.
- Adatto per un numero limitato di recensioni per ristorante.

Svantaggi:

- Se il numero di recensioni cresce troppo, il documento potrebbe diventare grande e difficile da gestire.
- Non ideale per query molto specifiche sulle recensioni (ad esempio, cercare una recensione in particolare).

- Document linking

In questo approccio, i ristoranti e le recensioni sono memorizzati in collezioni separate. Le recensioni contengono un riferimento al ristorante a cui appartengono (ad esempio un *restaurant_id*).

Documento ristorante:

```

{
  "restaurant_id": "1",
  "name": "A Casa Mia",
  "address": "Largo Lucio Lazzarino 1, Pisa"
}

```

Documento recensioni:

```
[
  {
    "comment_id": "1",
    "restaurant_id": "1",
    "comment": "Very Bad menu!"
  },
  {
    "comment_id": "2",
    "restaurant_id": "1",
    "comment": "The worst Restaurant in the World"
  }
]
```

Vantaggi:

- Più flessibile, poiché le recensioni sono gestite separatamente e possono crescere senza impattare direttamente sul documento del ristorante.
- Utile per query specifiche sulle recensioni o per elaborazioni su recensioni senza caricare l'intero ristorante.

Svantaggi:

- Richiede una sorta di join (applicato tramite codice) per combinare i dati del ristorante con le recensioni.
- Le query che coinvolgono ristorante e recensioni insieme possono essere più lente rispetto all'incorporazione.

5. Considera un namespace, accessibile attraverso la variabile “AppNameSpace” che include le seguenti entrate:

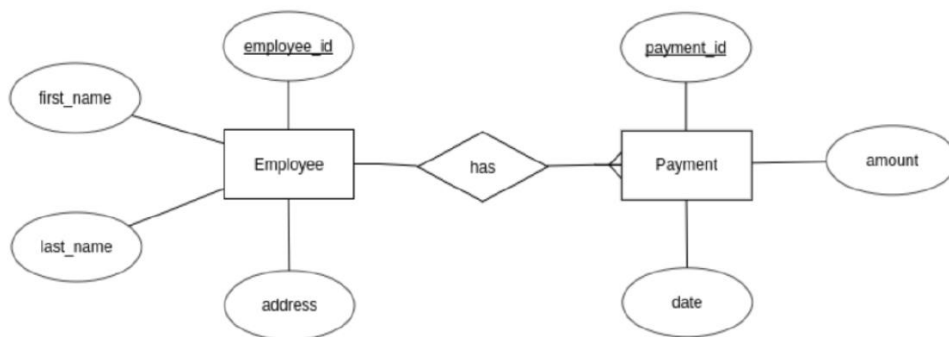
```
cust:1234123:firstName: "Pietro"  
cust:1234123:lastName: "Ducange"
```

Definisci i seguenti metodi di set e get:

```
getCustAttr(userId, attributeName)  
setCustAttr(userId, attributeName, value)
```

```
public static String getCustAttr(String userId, String attributeName) {  
    v_key = "cust:" + userId + ":" + attributeName; // Costruisce la chiave  
    return (AppNameSpace[v_key]);  
}  
  
public static void setCustAttr(String userId, String attributeName, String value) {  
    v_key = "cust:" + userId + ":" + attributeName; // Costruisce la chiave  
    AppNameSpace[v_key] = value; // Imposta il valore nel namespace  
}
```

6. Esempio delle slide “dal relazionale al keyvalue”, ma chiesto col document DB: traduci relazione 1 a m tra Impiegati e Pagamenti nel document DB. Di seguito sono presenti le due tabelle nel modello relazionale, definisci le query e produci il design del document DB in base alle query che hai definito.



In un modello relazionale possiamo definire le seguenti tabelle. La relazione 1 a molti è gestita usando una chiave esterna:

employee_id	first_name	last_name	address
1	John	Doe	New York
2	Benjamin	Button	Chicago
3	Mycroft	Holmes	London

FOREIGN KEY

payment_id	employee_id	amount	date
1	1	50,000	01/12/2017
2	1	20,000	01/13/2017
3	2	75,000	01/14/2017
4	3	40,000	01/15/2017
5	3	20,000	01/17/2017
6	3	25,000	01/18/2017

Definizione delle query: Identifichiamo le query principali per capire come strutturare il design del **Document DB**:

- **Query 1:** Recuperare tutti i dettagli di un dipendente dato il suo employee_id.
- **Query 2:** Recuperare tutti i pagamenti effettuati da un dipendente dato il suo employee_id.
- **Query 3:** Ottenere i dettagli di un pagamento specifico dato il suo payment_id.
- **Query 4:** Trovare il totale dei pagamenti effettuati da un dipendente.

Design del Document DB: Il design dipende da come sono distribuite e utilizzate le query. Possiamo optare per:

- **Approccio relazionale nidificato (embedding):** I dati sui pagamenti sono inclusi nel documento del dipendente, perché le query sono orientate a trovare pagamenti a partire da un dipendente.
- **Approccio referenziato:** Employee e Payment rimangono entità separate con riferimenti reciproci, utile per query indipendenti su Payment.

```
{
  "_id": "E001",
  "first_name": "John",
  "last_name": "Doe",
  "address": "123 Main St",
  "payments": [
    {
      "payment_id": "P001",
      "amount": 1500,
      "date": "2024-11-01"
    }
  ]
}
```

```

},
{
  "payment_id": "P002",
  "amount": 2000,
  "date": "2024-11-15"
}
]
}

```

- **Pro:**

- Le query 1, 2 e 4 sono molto efficienti perché tutti i dettagli sono in un unico documento.
- Non è necessaria una join, perché i pagamenti sono parte del documento dipendente.

- **Contro:**

- Meno adatto se i dati sui pagamenti devono essere aggiornati frequentemente o accessibili senza riferimento al dipendente.

- Se i pagamenti sono strettamente correlati ai dipendenti e raramente usati indipendentemente, l'approccio **embedding** è più efficiente.

- Se i pagamenti devono essere frequentemente aggiornati, interrogati separatamente o diventano molto numerosi, l'approccio **referenziato** è preferibile.

7. Esercizio di seguito

A University offers a collection of Master Programs (MPs). A student can be enrolled to an MP. An MP is composed by a set of courses. To get his/her degree, the student must pass all the exams of courses his/her specific MP. Provide the design of a document DB (collections and their structure) for storing the information regarding student career (including personal information) at the university and supporting the following functional requirements:

- Given a student, the application shall show all the courses of his/her MP
- Given a student, the application shall show all the exams that he/she passed (along with the associated marks)
- Given a specific MP, the application shall the average mark of the exams of each of its courses.

Please provide some justification supporting the DB design and show an example of the content of the database for the following student:

Name: Pippo

Surname: Pluto

DOB: July, 13, 2001

Master Program: AIDE

Courses to Attend: Large Scale, IoT, Cloud Computing, Data Mining

Passed Exams: Large Scale (25/30), Data Mining (22/30)

The Student Collection

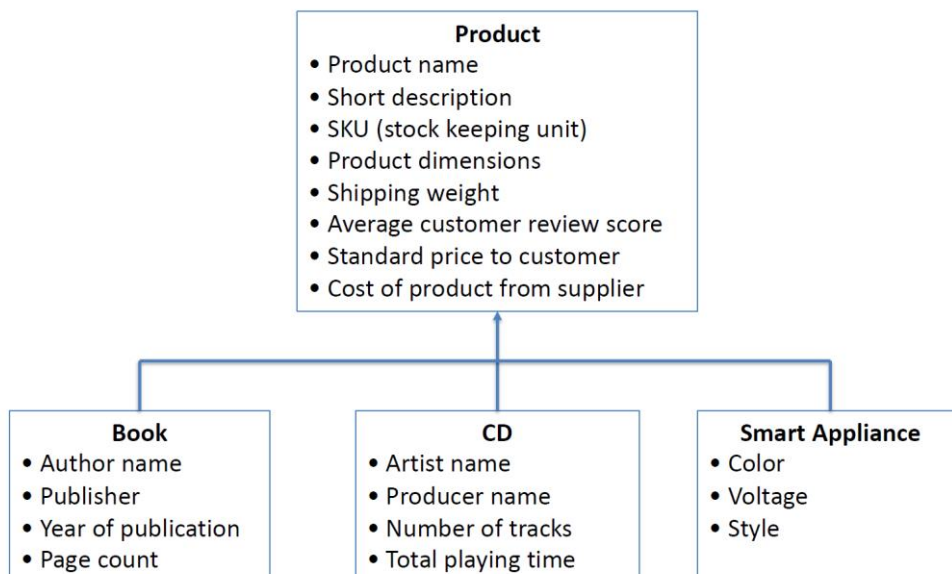
```
"name": "Pippo",
"surname": "Pluto",
"BOD": "July, 13, 2001",
"Master Program": "AIDE",
"Courses": [{"CourseName": "Large Scale", "mark": 27},
             {"CourseName": "IoT", "mark": -1},
             {"CourseName": "Data Mining", "mark": 22},
             {"CourseName": "Cloud Computing", "mark": -1}]
```

Questa collezione è basata sul document embedding, il quale è stato scelto perché è utile per velocizzare le prime due query in figura: infatti da questa struttura si possono estrarre le informazioni riguardo agli esami e allo studente stesso, facendo riferimento ad un'unica collezione.

```
{
  "Name": "AIDE",
  "Courses": [ {"CourseName": "Large scale", "avg_mark": 23}, ..., ... ]
}
```

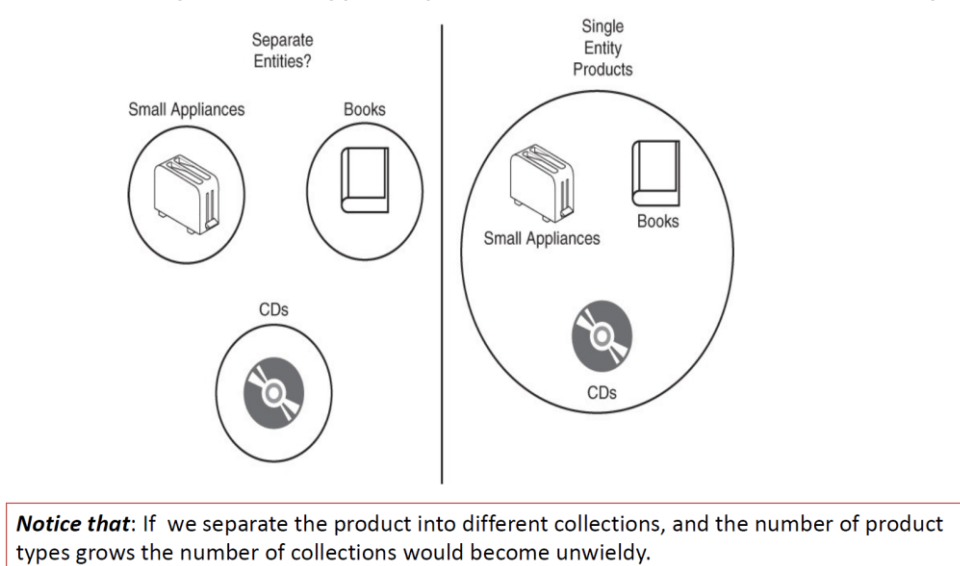
8. Table product + table books, table appliances, come li traduci in un document DB? In una singola collezione o in molteplici?

Follow the Definition of Queries



Our application might be able to answer the following queries:

- What is the average number of products bought by each customer?
- What is the range of number of products purchased by customer?
- What are the top 20 most popular products by customer state?
- What is the average value of sales by customer state?
- How many of each type of product were sold in the last 30 days?



Normalization or Denormalization?

Normalization helps *avoid* data *anomalies*, but it can cause *performance problems*.

With *normalized* data, we need *join operations*, which must be optimized for improving performances.

If we use *denormalized* data, we may introduce *redundancies* and cause anomalies.

On the other hand, we may *improve the performances* of the queries because we *reduce* the number of collections and *avoid join operations*.

Denormalization supports improving read operations when *indexes* are adopted.

Risposta:

Per tradurre le tabelle di "Product", "Book", "CD", e "Smart Appliance" in un document database (come MongoDB), ci sono due principali approcci da considerare:

- **1. Singola collezione:**

Tutti i tipi di prodotto possono essere memorizzati in una singola collezione chiamata, ad esempio, Products. In questo caso, i documenti avranno campi comuni a tutti i prodotti (ad esempio, ProductName, SKU) e campi specifici per ciascun tipo (ad esempio, AuthorName per i libri, Voltage per gli elettrodomestici).

Esempio di documenti nella collezione:

```
{
  "_id": "1",
  "ProductName": "Great Book",
  "ShortDescription": "A compelling novel.",
  "SKU": "12345",
  "ProductDimensions": "20x30x5 cm",
  "ShippingWeight": "1 kg",
  "AverageCustomerReviewScore": 4.5,
  "StandardPriceToCustomer": 19.99,
  "CostOfProductFromSupplier": 10.0,
  "Type": "Book",
  "AuthorName": "John Doe",
  "Publisher": "Fictional Press",
  "YearOfPublication": 2020,
  "PageCount": 300,
  "ArtistName": null,
  "ProducerName": null,
  "NumberOfTracks": null,
  "TotalPlayingTime": null,
  "Color": null,
  "Voltage": null,
  "Style": null
}
```

Vantaggi:

- Semplicità nella gestione di una sola collezione.
- Query su campi comuni (es. SKU o StandardPriceToCustomer) più semplici.

Svantaggi:

- La struttura dei documenti varia molto tra i tipi di prodotto, portando a molti campi nulli per alcuni tipi di prodotto.
- Crescente complessità nella gestione di query o indici su campi specifici: si ha una unica collezione più grande, ogni volta bisogna scorrere anche elementi non interessati, gli indici sono meno efficienti perché salvano anche i valori null

• 2. Collezioni multiple:

Creare collezioni separate per ciascun tipo di prodotto: ad esempio, una collezione Books, una collezione CDs e una SmartAppliances. Ogni collezione avrà i campi comuni come ProductName e SKU, ma anche campi specifici per il tipo.

Esempio:

- Collezione Books:

```
{
  "_id": "1",
  "ProductName": "Great Book",
  "ShortDescription": "A compelling novel.",
  "SKU": "12345",
  "ProductDimensions": "20x30x5 cm",
  "ShippingWeight": "1 kg",
  "AverageCustomerReviewScore": 4.5,
  "StandardPriceToCustomer": 19.99,
  "CostOfProductFromSupplier": 10.0,
  "AuthorName": "John Doe",
  "Publisher": "Fictional Press",
  "YearOfPublication": 2020,
  "PageCount": 300
}
```

- Collezione CDs:

```
{
  "_id": "2",
```

```
"ProductName": "Hit Album",  
"ShortDescription": "A top-charting album.",  
"SKU": "67890",  
"ProductDimensions": "15x15x1 cm",  
"ShippingWeight": "0.5 kg",  
"AverageCustomerReviewScore": 4.8,  
"StandardPriceToCustomer": 14.99,  
"CostOfProductFromSupplier": 7.0,  
"ArtistName": "Famous Artist",  
"ProducerName": "Top Producer",  
"NumberOfTracks": 12,  
"TotalPlayingTime": "45 min"  
}
```

Vantaggi:

- Ogni collezione ha documenti con struttura uniforme.
- Più facile gestire campi specifici e ottimizzare le query per ciascun tipo.

Svantaggi:

- Query sui campi comuni richiedono l'aggregazione tra collezioni.
- Più complesso da gestire se si aggiungono nuovi tipi di prodotto.

- **Quale approccio scegliere?**

Dipende dalle esigenze specifiche del progetto:

- Se il focus è sulla semplicità e la maggior parte delle query riguarda campi comuni, una singola collezione è più adatta.
- Se ogni tipo di prodotto ha molti campi specifici e poche query incrociate tra tipi di prodotto, è preferibile usare collezioni multiple.

Esempi dalle slide per la parte di esercizi

Esempio 1 – Query nel columnar DB partendo da relational DB

Partiamo dal seguente database relazionale:

Customers (*one-to-many relationship con Orders*)

- `CustomerID`
- `Name`
- `Address`

Orders (*one-to-many relationship con Customers*)

- `OrderID`
- `CustomerID`
- `Date`
- `Amount`

Products (*many-to-many relationship con Orders attraverso OrderProducts*)

- `ProductID`
- `Name`
- `Price`

OrderProducts (*linking table per la many-to-many relationship*)

- `OrderID`
- `ProductID`
- `Quantity`

Vogliamo arrivare al columnar, che ricordiamo, ha i dati salvati in colonne invece che righe.

Soluzione:

La query che ci interessa è: *contare il numero di prodotti comprati da un certo utente*. Nei database columnar, l'architettura a colonne si adatta molto bene a questa query, dato che è possibile accedere selettivamente alle colonne coinvolte nel calcolo senza leggere dati non necessari.

OrderProducts Table

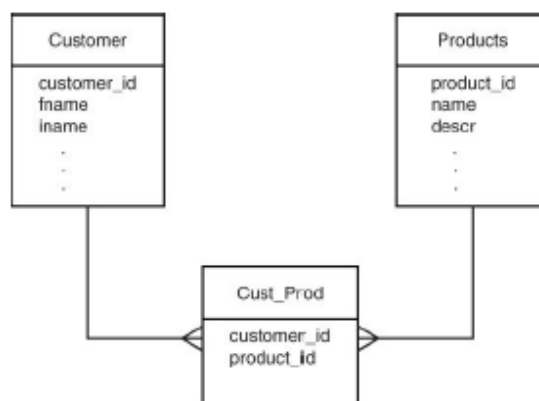
- Row key: *CustomerID#OrderID#ProductID*
- Column family: called “*details*”

Si effettua la query nella tabella OrderProducts table usando un prefisso per scannerizzare: con CustomerID# si ottengono tutti gli ordini di uno specifico cliente, poi si aggregano le quantità di prodotti.

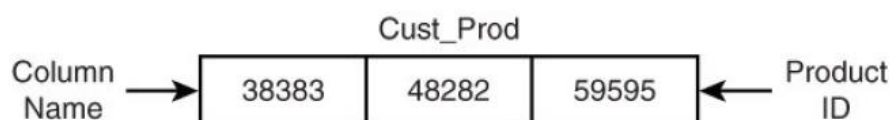
Row Key	details
Customer1#OrderA#ProductX	2
Customer1#OrderA#ProductY	1
Customer1#OrderB#ProductX	3
Customer2#OrderC#ProductZ	5
Customer2#OrderD#ProductY	2

Esempio 2 – Evitare le join nelle relazioni many to many Columnar DB

Creiamo una struttura dati derivata dalle due in alto, che in generale si ottiene attraverso una operazione di join.



Per ogni customer si ha una lista di ID di prodotti, è una struttura ridondante: questa struttura è una column family dove ogni colonna è un valore (il nome della colonna può salvare un valore).



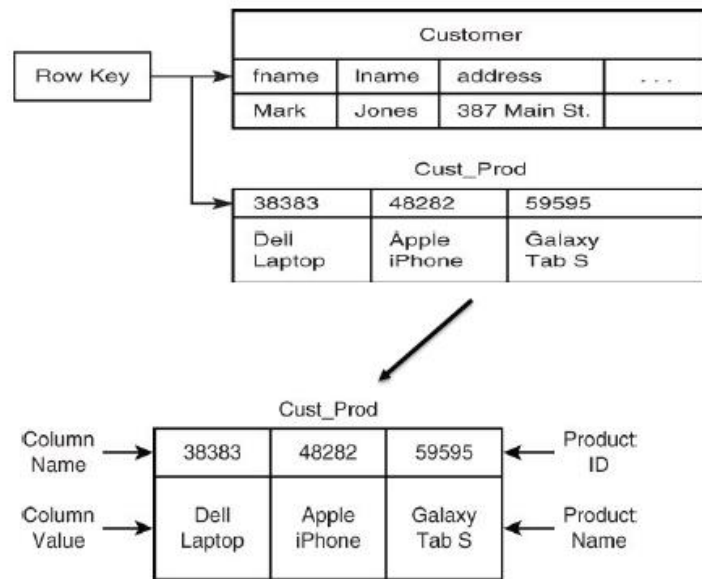
Relational DB

We use 3 tables, including a join table.

Column DB

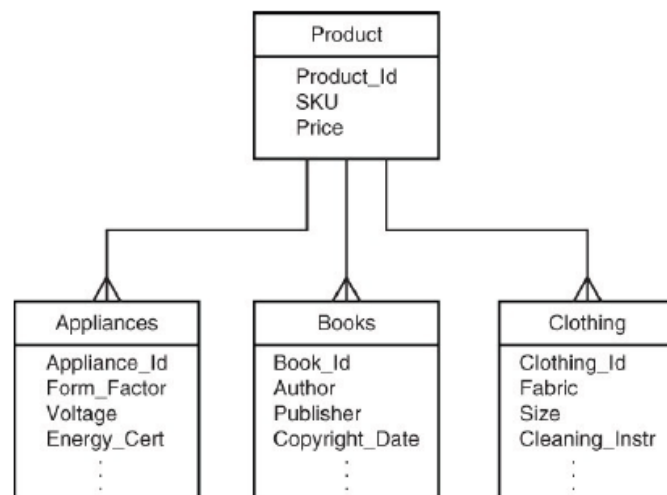
In order to avoid join operations, each customer includes a **set of column names** that correspond to purchased products (Ids).

Attraverso la row key posso andare a recuperare non solo i dati del customer ma anche i suoi prodotti comprati. In questo caso i nomi delle colonne sono valori e ogni colonna non ha valori (*valueless*).



Poiché il valore di ogni colonna non viene utilizzato per nient'altro, possiamo memorizzare il nome del prodotto lì. Questo può evitare l'accesso alla tabella dei prodotti, per esempio se vogliamo produrre un report che elenca i prodotti acquistati da un cliente. Conservare una copia del nome del prodotto nella tabella dei clienti aumenterà la quantità di spazio di archiviazione utilizzato (miglioriamo le prestazioni di lettura). In questo caso ogni colonna ha un valore, rispetto al caso precedente. Questi appena descritti sono due modi per definire una relazione many to many al fine di evitare operazioni di join.

Esempio 3 – Modellizzare una entità in una singola riga



Una singola istanza di un'entità, come un cliente particolare o un prodotto specifico, dovrebbe avere tutti i suoi attributi in una singola riga. In questo caso, tipico dei columnar DB, alcune righe memorizzano più valori di colonna rispetto ad altre.

La figura mostra l'organizzazione classica in un DB relazionale di un'entità che può assumere diverse "forme". Per garantire l'atomicità dell'operazione nel columnar DB, la stessa entità nella figura deve essere organizzata in una sola tabella con 4 column families.



Questa immagine mostra come un column DB potrebbe rappresentare l'entità "Prodotto" con una singola riga che include tutte le colonne rilevanti, anche se alcune saranno riempite solo per categorie specifiche. Ogni riga ha un Row id unico, seguito da colonne suddivise per tipo di prodotto (Product_columns, Appliance_columns, ecc.). Solo le colonne rilevanti per una certa categoria di prodotto saranno valorizzate, mentre le altre resteranno vuote.

Atomicità: L'atomicità è garantita a livello di riga, evitando la necessità di aggiornamenti su più tabelle.

Esempio 4 – Evitare strutture dati complesse come valori nei columnar

Qualsiasi tipo di struttura dati può essere memorizzata in un valore di una colonna, ad esempio un file JSON:

```
{
  "customer_id":187693,
  "name": "Kiera Brown",
  "address" : {
    "street" : "1232 Sandy Blvd.",
    "city" : "Vancouver",
    "state" : "Washington",
    "zip" : "99121"
  },
  "first_order" : "01/15/2013",
  "last_order" : " 06/27/2014"
}
```

Nota che:

- L'utilizzo di colonne separate per ogni attributo semplifica l'applicazione delle funzionalità del database agli attributi (come l'indexing).
- La separazione degli attributi in colonne individuali consente di utilizzare diverse column families, se necessario.

Esempio 5 – Indici nei columnar

In un columnar DB, un indice viene utilizzato per trovare rapidamente le righe che fanno riferimento a quello specifico valore di una specifica colonna.

Abbiamo due tipi di indici:

- Gli indici primari sono indici sulle row key
- Gli indici secondari sono indici creati su uno o più valori di una colonna.

Se vogliamo velocizzare una query come questa:

```
SELECT * FROM Tab WHERE state = "OR" AND Lname = "TR"
```

Dovremmo mettere un indice secondario su *state* e *Lname*.

Gli indici secondari vengono creati per colonne specifiche e memorizzano l'associazione tra il valore della colonna e la posizione della riga.

Gli indici secondari possono migliorare l'efficienza della query ma possono anche introdurre un sovraccarico aggiuntivo in termini di spazio e prestazioni durante gli aggiornamenti dei dati.

Esempio di un Indice secondario su state: Questo indice tiene traccia di tutti i valori presenti nella colonna *state* e della posizione di ogni riga con ciascun valore. Ad esempio, per *state* = "OR", l'indice conserva un elenco delle posizioni di tutte le righe dove *state* è "OR".

Creazione di un Indice Composto

Un'alternativa è creare un **indice composto** su entrambe le colonne *state* e *Lname*. Un indice composto crea un ordinamento basato sulla combinazione dei valori nelle due colonne. In questo caso, l'indice organizzerebbe prima i dati in base al valore di *state* e, tra quelli con lo stesso valore di *state*, ordinerebbe in base a *Lname*.

Ad esempio, un indice composto su (*state*, *Lname*) potrebbe apparire così:

state	Lname	RowID
OR	TR	12
OR	TR	37
OR	TX	14
WA	TR	23

When avoiding to use automatically managed indexes

Column Family				
Name	Address	Opt In?	City	
		Y		
		Y		
		N		
		Y		
		N		
		.		
		.		
		Y		
		N		

Only Two Distinct Values

An index will probably not help much, especially if there are ***roughly equal numbers*** of each value.

Column Family				
Name	Address	City	State	Email
				raiken@gmail.com
				iman123@gmail.com
				clans37@yahoo.com
				marypdx@gmail.com
				gwashtington@aol.com
				kcameron@future.com
				info@mybbiz.com
				-
				-
				-
				-
				-

Many Distinct Values

Indexes may not help much here because the index will have to ***maintain so much data*** it could take ***too much time*** to search the index and retrieve the data.

[illegible]Sparsely
Populated

Too few values associated to a specific column. It is not worth creating and managing an index data structure for this attribute.

Esempio 6 – Graph DB vs RDB

Esempio di query: elenca tutti i corsi a cui è iscritto uno studente in particolare.

Nei modelli relazionali, abbiamo bisogno di una procedura di join.

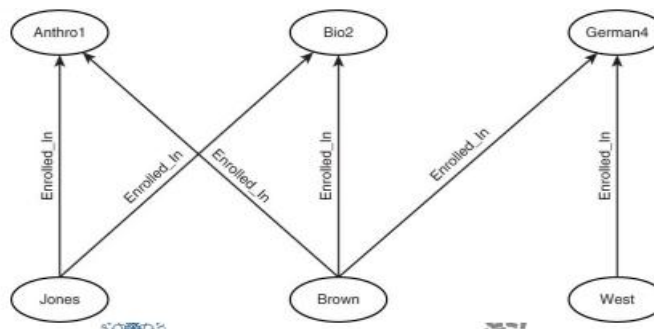
The diagram illustrates a data model with three tables: Students, Enrollment, and Courses. The Enrollment table is the central fact table, and Students and Courses are dimension tables. The relationships are as follows:

- Students** (ID, Name) is connected to **Enrollment** (ID, Course) via a one-to-many relationship on the ID field.
- Enrollment** (ID, Course) is connected to **Courses** (Course, Title) via a one-to-many relationship on the Course field.

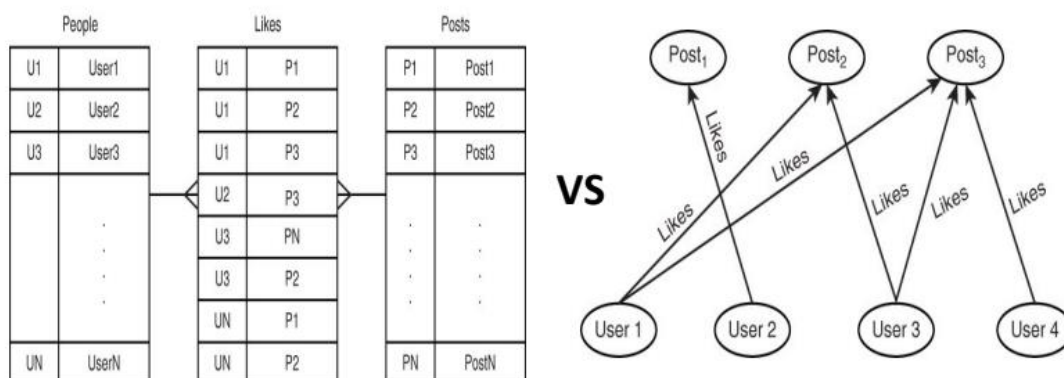
The data in the tables is as follows:

Students		Enrollment		Courses	
123	Jones	123	Anthro1	Anthro1	Intro. to Anthropology
278	Brown	123	Bio2	Bio2	Evolutionary Biology
789	West	278	Bio2	German4	German Literature
.	.	278	Anthro1	.	.
.	.	789	German4	.	.
.	.	.	.	.	.
.	.	.	.	.	.

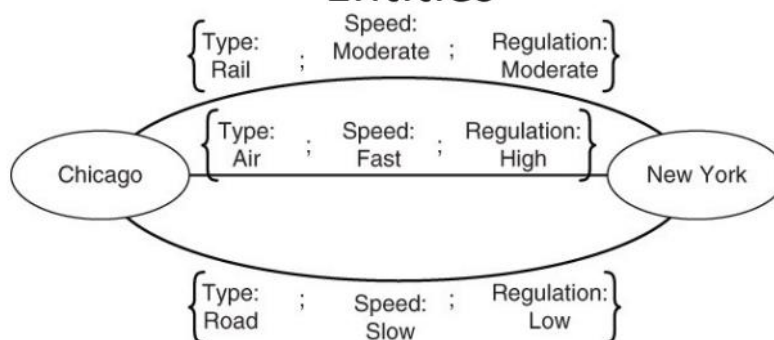
Gli archi tra studenti e corsi ci consentono di interrogare rapidamente il database.



Esempio 7 – Many to Many nei Graph DB



Modeling Multiple Relations between Entities



Le relazioni possono avere delle proprietà, e in un Graph DB questa cosa è facile da implementare, perché posso avere più tipi di archi (specificando diversi valori delle proprietà negli archi) e ogni tipo di arco può avere valori delle proprietà diversi.

Query tipiche su un Graph database:

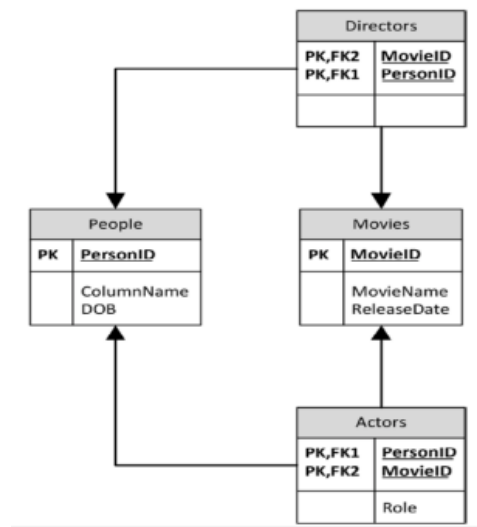
- Quanti archi tra due nodi con determinate proprietà?
- Quanti archi sono collegati a un nodo.
- Vicinanza e centralità di un nodo?
- Un nodo è un collo di bottiglia?
- Se un nodo viene rimosso, quali parti del grafico diventano disconnesse?

Esempio 8 – un semplice Graph DB

Hollywood DB: Supponiamo di progettare un DB per gestire film, attori e registi. Supponiamo di offrire i seguenti servizi agli utenti:

1. Trovare tutti gli attori che sono mai apparsi come co-protagonisti in un film con un attore specifico (ad esempio: Keanu Reeves).
1. Trovare tutti i film e gli attori diretti da un regista specifico (ad esempio Andy Wachowski).
2. Trovare tutti i film in cui ha recitato un attore specifico, insieme a tutti i co-protagonisti e registi.

Di seguito è indicato il modello relazionale:



Problemi:

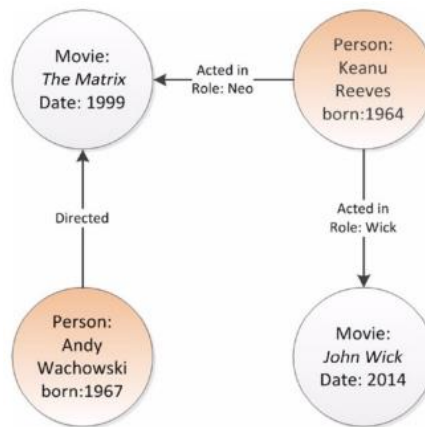
- SQL non ha la sintassi per eseguire facilmente l'attraversamento del grafico, in particolare gli attraversamenti in cui la profondità è sconosciuta o illimitata.
- Le prestazioni peggiorano rapidamente man mano che attraversiamo il grafico. Ogni livello di attraversamento aggiunge significativamente tempo di risposta a una query specifica

Di seguito, mostriamo una query SQL per trovare tutti gli attori che hanno mai lavorato con Keanu Reeves:

```
1 SELECT p2.personname, m1.movieName
2 FROM people p1
3 JOIN actors a1 ON (p1.personid = a1.personid)
4 JOIN movies m1 ON (a1.movieid = m1.movieid)
5 JOIN actors a2 ON (a2.movieid = m1.movieid)
6 JOIN people p2 ON (p2.personid = a2.personid)
7 WHERE p1.personname = 'Keanu Reeves';
```

Questa è una query molto pesante per i database relazionali.

Di seguito è rappresentato il modello con il Graph DB:



```

neo4j-sh (?)$ MATCH (kenau:Person {name:"Keanu Reeves"})
-[:ACTED_IN]->(movie)<-[:ACTED_IN]-(coStar)
RETURN coStar.name;

```

Esempio 9 – Social Network

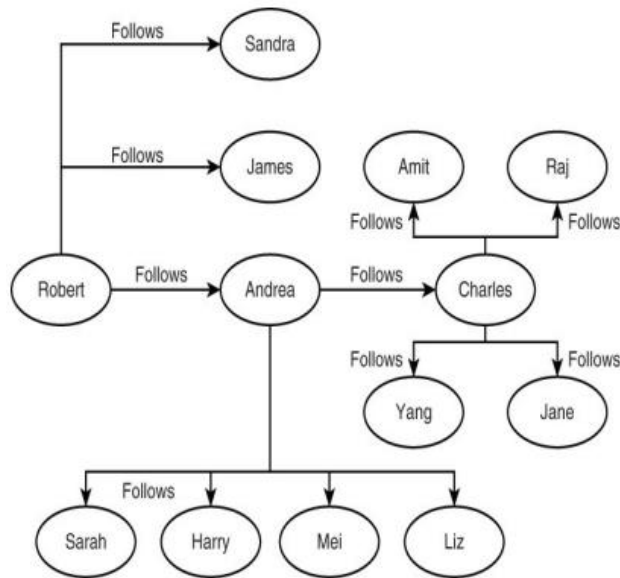
Definisci i principali casi d'uso:

1. Iscriviti e abbandona il sito.
2. Segui uno sviluppatore.
3. Pubblica domande per esperti in un'area specifica.
4. Suggerisci nuove connessioni con altri sviluppatori in base a interessi condivisi.
5. Classifica i membri in base al numero di connessioni, post e risposte.

Definiamo le entità e le loro proprietà:

Developer	Post
• Name • Location • NoSQL databases used • Years of experience • Area of interests	• Date of creation • Topic Keywords • Post type ○ Example ○ Question ○ Tip ○ News ○ ... • Title of the post • Body of the post

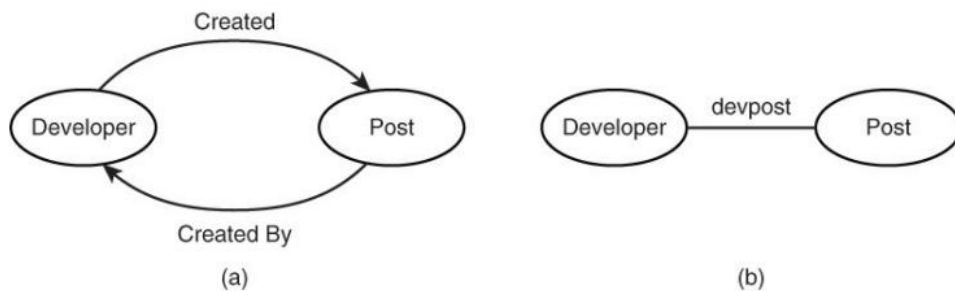
In generale bisogna focalizzarsi sulle relazioni tra entità più utilizzate nelle query se il numero di entità cresce, come la “Sviluppatore segue lo sviluppatore”:



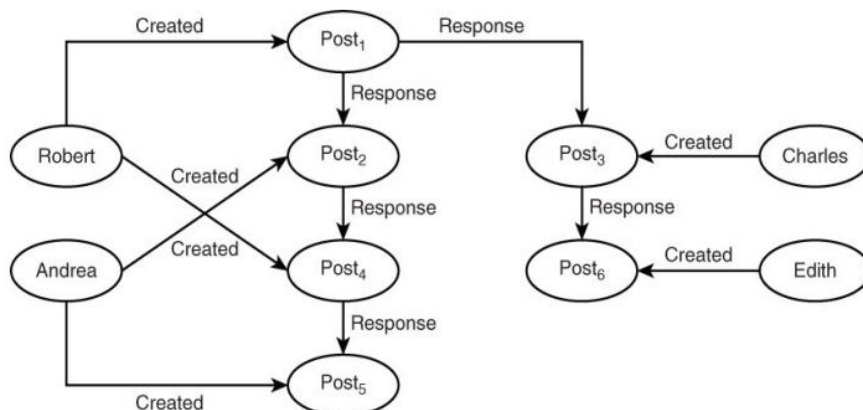
Se uno sviluppatore segue un altro sviluppatore, è ragionevole che sia interessato a leggere i suoi post. La profondità del percorso, ovvero quante frecce attraversare per identificare uno sviluppatore seguito, è un parametro che può essere definito e gestito a livello di codice applicativo. Esploriamo la prima relazione (molto-a-molti): Uno sviluppatore può seguire un altro sviluppatore, quindi il nome della prima relazione potrebbe essere "follows".

Se uno sviluppatore X segue un altro sviluppatore Y, allora X è interessato ai post di Y.

Relazioni tra developer e post (1 a molti):



Nella figura soprastante, mostriamo uno sviluppatore e un post con due archi diretti (a) o un arco non diretto. Relazione post-post (molti a molti):

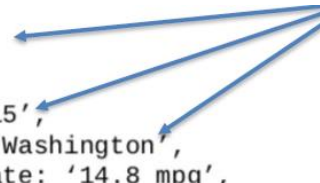


Un post può essere creato in risposta a un altro post!

Esempio 10 – Data Modeling

I camion di una azienda devono trasmettere posizione, consumo di carburante e altre metriche ogni tre minuti a un database di gestione (relazione uno-a-molti tra un camion e i dettagli trasmessi). Potremmo prendere in considerazione la possibilità di generare un nuovo documento da aggiungere al DB per ogni trasmissione di dati.

```
{
  truck_id: 'T87V12',
  time: '08:10:00',
  date : '27-May-2015',
  driver_name: 'Jane Washington',
  fuel_consumption_rate: '14.8 mpg',
  ...
}
```



Le parti indicate dalle frecce sono informazioni ripetute in ogni documento.

Alla fine della giornata, il DB includerà 200 nuovi documenti per ogni camion (consideriamo 20 trasmissioni all'ora, 10 ore di lavoro). In questo caso potrei considerare la soluzione del document embedding:

```
{
  truck_id: 'T87V12',
  date : '27-May-2015',
  driver_name: 'Jane Washington',
  operational_data:
    [
      {time : '00:01',
       fuel_consumption_rate: '14.8 mpg',
       ...},
      {time : '00:04',
       fuel_consumption_rate: '12.2 mpg',
       ...},
      {time : '00:07',
       fuel_consumption_rate: '15.1 mpg',
       ...},
      ...]
}
```

Ma bisogna fare attenzione a potenziali problemi di performance! Questo perché quando un documento è creato il DBMS alloca un certo ammontare di spazio per il documento, se cresce più dello spazio allocato, il DBMS lo deve rilocare. Inoltre, il DBMS deve liberare lo spazio precedentemente allocato. Questi due step sono costosi in termini di performance. Come si risolve? Si alloca uno spazio sufficiente sin dall'inizio quando il documento è creato.

```
{truck_id: 'T8V12'
 date: '27-May-2015'
 operational_data:
  [{time: '00 : 00',
   fuel_consumption_rate: 0.0}
   {time: '00 : 00',
   fuel_consumption_rate: 0.0}
   .
   .
   .
   {time: '00 : 00',
   fuel_consumption_rate: 0.0}
  ]
}
```

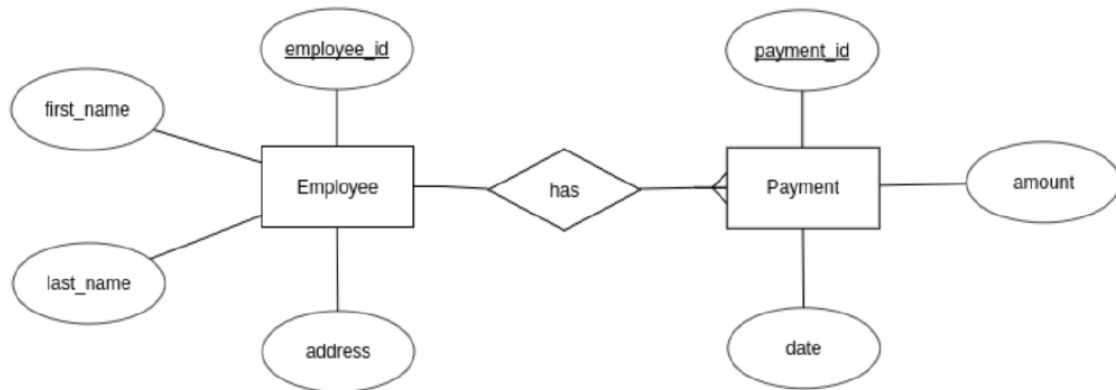
200 Embedded Documents with Default Values



Si potrebbe usare questa soluzione. In generale, bisogna controllare la vita intera del documento e pianificare se possibile le strategie di gestione della crescita.

Esempio 11 – Da relational DB a Key value DB

Consideriamo le seguenti strutture dati:



In un modello relazionale possiamo definire le seguenti tabelle. La relazione 1 a molti è gestita usando una chiave esterna:

employee_id	first_name	last_name	address
1	John	Doe	New York
2	Benjamin	Button	Chicago
3	Mycroft	Holmes	London

payment_id	employee_id	amount	date
1	1	50,000	01/12/2017
2	1	20,000	01/13/2017
3	2	75,000	01/14/2017
4	3	40,000	01/15/2017
5	3	20,000	01/17/2017
6	3	25,000	01/18/2017

FOREIGN KEY

Ora, vogliamo tradurre la modellazione dei dati da un relational model a un key-value model. Tieni presente che le chiavi incorporano informazioni riguardanti: *Entity Name*, *Entity Identifier* ed *Entity Attributes*. Quindi, possiamo tradurre la tabella *Employees* come segue:

employee_id	first_name	last_name	address
1	John	Doe	New York
2	Benjamin	Button	Chicago
3	Mycroft	Holmes	London



employee:\$employee_id:\$attribute_name = \$value

```
employee:1:first_name = "John"
employee:1:last_name = "Doe"
employee:1:address = "New York"
```

```
employee:2:first_name = "Benjamin"
employee:2:last_name = "Button"
employee:2:address = "Chicago"
```

```
employee:3:first_name = "Mycroft"
employee:3:last_name = "Holmes"
employee:3:address = "London"
```

Se una voce non ha un valore per un attributo (ad esempio l'indirizzo), è un errore mettere questa associazione con un valore nullo. La soluzione: non mettere un'informazione se non è associata a un valore.

Come ulteriore passaggio, dobbiamo tradurre la tabella dei pagamenti e gestire la relazione uno-a-molti. In questo caso, possiamo definire la seguente configurazione chiave-valore:

payment:\$payment_id:\$employee_id:\$attribute_name = \$value

La tabella dei pagamenti può essere tradotta come segue:

payment_id	employee_id	amount	date
1	1	50,000	01/12/2017
2	1	20,000	01/13/2017
3	2	75,000	01/14/2017
4	3	40,000	01/15/2017
5	3	20,000	01/17/2017
6	3	25,000	01/18/2017



```
payment:1:1:amount = "50000"  
payment:1:1:date = "01/12/2017"  
  
payment:2:1:amount = "20000"  
payment:2:1:date = "01/13/2017"  
  
payment:3:2:amount = "75000"  
payment:3:2:date = "01/14/2017"  
  
payment:4:3:amount = "40000"  
payment:4:3:date = "01/15/2017"  
  
payment:5:3:amount = "20000"  
payment:5:3:date = "01/17/2017"  
  
payment:6:3:amount = "25000"  
payment:6:3:date = "01/18/2017"
```

Alla fine del processo di traduzione, i dati saranno organizzati in un unico bucket come segue:

```
employee:1:first_name = "John"  
employee:1:last_name = "Doe"  
employee:1:address = "New York"  
  
employee:2:first_name = "Benjamin"  
employee:2:last_name = "Button"  
employee:2:address = "Chicago"  
  
employee:3:first_name = "Mycroft"  
employee:3:last_name = "Holmes"  
employee:3:address = "London"  
  
payment:1:1:amount = "50000"  
payment:1:1:date = "01/12/2017"  
  
payment:2:1:amount = "20000"  
payment:2:1:date = "01/13/2017"  
  
payment:3:2:amount = "75000"  
payment:3:2:date = "01/14/2017"  
  
payment:4:3:amount = "40000"  
payment:4:3:date = "01/15/2017"  
  
payment:5:3:amount = "20000"  
payment:5:3:date = "01/17/2017"  
  
payment:6:3:amount = "25000"  
payment:6:3:date = "01/18/2017"
```

Esempio 12 – Gestire range di valori key value

cust : p_date : p_id : custId

- La data di acquisto (p_date), per ottenere rapidamente informazioni sul cliente che ha effettuato quell'acquisto specifico.
- L'ID prodotto (p_id).
- Il customer ID (custId).

Query: recupera tutti i clienti che hanno effettuato un acquisto in una data specifica.
Possiamo definire chiavi associate ai clienti che hanno acquistato

Prodotti in una data specifica come nel seguente esempio:

```
cust:061514:1:custId  
cust:061514:2:custId  
cust:061514:3:custId  
cust:061514:4:custId
```

Questo tipo di chiave è utile per interrogare intervalli di chiavi perché puoi scrivere facilmente una funzione per recuperare un intervallo di valori.

La seguente funzione recupera un elenco di customerID che hanno effettuato acquisti in una data specifica:

```
define getCustPurchByDate(p_date)  
    v_custList = makeEmptyList();  
    v_rangeCnt = 1;  
  
    v_key = 'cust:' + p_date + ':' + v_rangeCnt +  
        ':custId';  
    while exists(v_key)  
        v_custList.append(myAppNS[v_key]);  
        v_rangeCnt = v_rangeCnt + 1;  
        v_key = 'cust:' + p_date + ':' + v_rangeCnt +  
            ':custId';  
  
    return(v_custList);
```

1. Inizialmente creiamo un elenco vuoto di clienti.
2. Inizializziamo l'intervallo (*v_rangeCnt*) a 1, è il primo elemento dell'elenco (è il contatore degli acquisti).
 - a. 1 significa: "quel cliente è stato il primo a effettuare un acquisto in quella data".
3. Creiamo la chiave che ci consente di recuperare i dati che desideriamo (il *custId* va specificato in maniera precisa, nella figura è considerato generico)
4. Ciclo:
 - a. Controlliamo se la chiave esiste (se c'è una voce) con *exist(v_key)*
 - b. Se la chiave esiste, recuperiamo i valori *.append(myAppNS[v_key])*
 - c. Incrementiamo *v_rangeCnt* e aggiorniamo la chiave.

A volte durante l'esame viene richiesto di scrivere un codice come questo.

L'unico modo per cercare i valori è tramite la chiave, quindi non è l'ideale quando devo fare molte query di intervallo.

Esempio 13- Valori complessi key value

Considera la seguente funzione che prende sia il nome che l'indirizzo:

```
define getCustNameAddr(p_id)
    v_fname = getCustAttr(p_id,'fname');
    v_lname = getCustAttr(p_id,'lname');
    v_addr = getCustAttr(p_id,'addr');
    v_city = getCustAttr(p_id,'city');
    v_state = getCustAttr(p_id,'state');
    v_zip = getCustAttr(p_id,'zip');
    v_fullName = v_fname + ' ' + v_lname;
    v_fullAddr = v_city + ' ' + v_state + ' ' + v_zip;
    return(makeList(v_fullName, v_fullAddr);
```

La funzione *getCustNameAddr* effettua sei accessi al database (sul disco) utilizzando la funzione *getCustAttr*. Per velocizzare l'esecuzione della funzione, dovremmo ridurre il numero di volte in cui lo sviluppatore deve chiamare *getCustAttr* o memorizzare i dati nella cache in memoria. Possiamo memorizzare insieme i valori degli attributi comunemente utilizzati come segue:

```
cstMgtNS[cust: 198277:nameAddr] = '{ 'Jane Anderson' ,
    '39 NE River St. Portland, OR 97222'}
```

I key-value DB solitamente memorizzano l'intero elenco insieme in un blocco di dati, quindi verrà effettuato l'accesso a un solo blocco sul disco, anziché a sei.

Bisogna però fare anche attenzione a non avere strutture dati troppo complesse come valori:

```
{
  'custFname': 'Liona',
  'custLname': 'Williams',
  'custAddr' : '987 Highland Rd',
  'custCity' : 'Springfield',
  'custState': 'NJ',
  'custZip'  : 21111,
  'ordItems' [
    {
      'itemID' : '85838A',
      'itemQty' : 2 ,
      'descr' : 'Intel Core i7-4790K Processor
        (8M Cache,
        4.40 GHz)',
      'price:' : $325.00
    } ,
    {
      'itemID' : '38371R',
      'itemQty' : 1 ,
      'descr' : 'Intel BOXDP67BGB3 Socket 1155, Intel
        P67',
        CrossFireX & SLI SATA3&USB3.0, A&GbE, ATX
        Motherboard',
      'price' : $140.00
    } ,
    {
      'itemID' : '10484K',
      'itemQty' : 1,
      'descr' : 'EVGA GeForce GT 740 Superclocked Single
        Slot 4GB
        DDR3 Graphics Card'
      'price': '$201.00'
    } ,
  ]
}
```

Questa intera struttura può essere archiviata sotto una chiave ordine, come 'ordID:781379'.

Il vantaggio di usare una struttura come questa è che molte delle informazioni sugli ordini sono disponibili con una singola ricerca chiave. Man mano che la struttura cresce in dimensioni, il tempo necessario per leggere e scrivere i dati può aumentare, perché il valore verrà archiviato in più di un blocco di memoria.

In generale, se dobbiamo usare una struttura complessa per il DB della nostra applicazione, è meglio passare ad architetture diverse, come i document DB.

Esempio 14 – Mobile app Key value

Consideriamo un'applicazione mobile utilizzata per tracciare le spedizioni dei clienti.

Le seguenti informazioni di configurazione su ciascun cliente sono archiviate in un database centralizzato:

- Nome del cliente e numero di conto
- Valuta predefinita per le informazioni sui prezzi
- Attributi di spedizione da visualizzare nella dashboard di riepilogo
- Avvisi e preferenze di notifica
- Opzioni dell'interfaccia utente, come schema di colori e font preferiti

La maggior parte delle operazioni sul DB saranno operazioni di lettura

Convenzione di denominazione per chiavi → *tipo di entità: numero di conto*

Entità identificate:

- Informazioni sul cliente, abbreviato "cust"
- Opzioni di configurazione della dashboard, abbreviato "dshb"
- Specifiche di avvisi e notifiche, abbreviato "alrt"
- Configurazioni dell'interfaccia utente, abbreviato "ui"

Customer Information:

```
TrackerNS['cust:4719364'] = {'name':'Prime Machine, Inc.',  
                              'currency':'USD'}
```

Dashboard Configuration options:

```
TrackerNS['dash:4719364'] =  
    {'shpComp','shpState','shpDate','shpDelivDate'}
```

Dashboard Configuration Options

- Ship to company (shpComp)
- Ship to city (shpCity)
- Ship to state (shpState)
- Ship to country (shpCountry)
- Date shipped (shpDate)
- Expected date of delivery (shpDelivDate)
- Number of packages/containers shipped (shpCnt)
- Type of packages/containers shipped (shpType)
- Total weight of shipment (shpWght)
- Note on shipment (shpNotes)

Alert and notification specifications:

```
TrackerNS[alrt:4719364] =  
  { altList :  
    { 'jane.washington@primemachineinc.com', 'pickup' },  
    { '(202) 555-9812', 'delay' }  
  }
```

User interface configuration options:

```
TrackerNS[ui :4719364] = { 'fontName': 'Cambria',  
  'fontSize': 9,  
  'colorScheme' : 'default'  
}
```

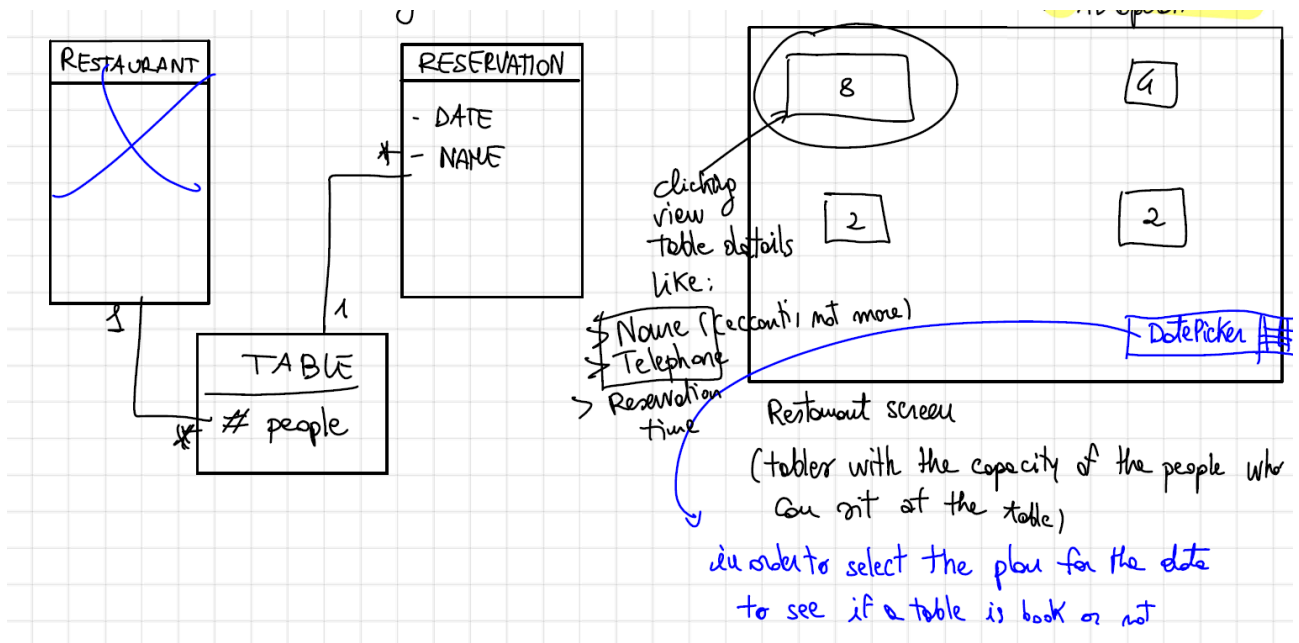
Esempio 15 – Caso del ristorante

Vogliamo sfruttare il sistema per prendere e mantenere le prenotazioni. Supponiamo che in una data "aaaa-mm-gg" possiamo avere solo una prenotazione per tavolo.

Per ogni tavolo abbiamo il numero massimo di persone che possono sedersi in quel tavolo, è un valore fisso per ogni tavolo.

Come dovrebbe funzionare il sistema:

- Se fai clic su un tavolo prenotato hai alcune informazioni sul nome e il numero di telefono del cliente, il DB mantiene queste informazioni solo per il tempo necessario per la prenotazione e non in modo persistente, sullo schermo abbiamo anche la data.
- Le operazioni di scrittura riguardano i tavoli, possiamo selezionare un tavolo e inserire i posti, il nome del cliente, il numero di telefono e la data.



Perché possiamo usare un key-value DB?

- Modello molto semplice: in pratica c'è solo l'entità "Table" e l'entità "Reservation"
- Relazioni 1 a n
- Nessuna analytic difficile

Come possiamo formattare la chiave?

Res : \$id_table : \$date : attributo

Non possiamo avere due prenotazioni nella stessa tabella alla stessa data, quindi quei due valori (indicati con \$) possono identificare la prenotazione (senza sfruttare un reservation_id). In questo caso la complessità è costante, si può capire la prenotazione e se il tavolo è libero o meno in quella data.

Quali attributi posso inserire?

Dobbiamo mantenere la descrizione della prenotazione, così possiamo aggiungere un nuovo attributo chiamato: \$res_des.

\$res_des può essere tipo: {Nome cliente, Numero di cellulare, Numero di persone}

Alla fine, abbiamo ottenuto queste entità

Prenotazione: \$id_table: \$date: \$res_des

Tavolo: \$id_table: \$maximumPeople

Attenzione ai requisiti! Il sistema è usato SOLO per creare e vedere le prenotazioni, per controllare una prenotazione ho troppi dettagli.

Posso organizzare il database come un calendario?

Quindi, aggiungo un'altra chiave, che mostra informazioni diverse:

Res: \$id_table: \$date: \$status

Se *status* = 0 allora la tabella per quella data è disponibile; altrimenti, l'attributo mi dice il numero di persone della prenotazione.

Esempio 16 – Ecommerce

Considerati i requisiti dell'applicazione di e-commerce, progettare la struttura del database dei documenti.

In particolare, identificare le collezioni e una possibile struttura di ciascuna di esse.

Prestare particolare attenzione ai seguenti requisiti funzionali:

1. Il sistema deve consentire a un gestore di trovare/sfogliare gli ordini di un cliente selezionato (in particolare, quelli più recenti). Considerare che una vista semplificata di ogni ordine sarà disponibile per l'utente
2. Il sistema deve offrire al responsabile di un'azienda un'analisi dei prodotti, vale a dire
 - a) Per mostrare i prodotti più venduti per categoria (in un intervallo di tempo)
 - b) Per mostrare i prodotti meno venduti per categoria (in un intervallo di tempo)
3. Il sistema deve offrire al responsabile di un'azienda analisi sui clienti, vale a dire:
 - a) Per mostrare gli utenti più attivi (in termini di numero di ordini consegnati in un intervallo di tempo)
 - b) Per mostrare gli utenti inclini a spendere più di altri (ad esempio, la classifica degli utenti in termini di denaro speso in un intervallo di tempo).

.ME DIC

User{

_id:

year:

FirstName:

LastName:

--:,

--:,

Type:,

Company: {id:, logo_url:, website}

RecentOrderList:[

{orderId:1,

date:,

ProductList:[{Name:, Description:, Category:, Quantity:, ...}, {Name:, Description:, Category:, Quantity:, ...}],

Status:,

--;

--;

TotalPrice:.},

{orderId:2,

date:.

ProductList:[{Name:., Description:., Category:., Quantity:., ...}, [Name:., Description:., Category:., Quantity:., ...}],

Status:.

--;

--;

TotalPrice:.},

{},.

orderList:[{orderId:1, Date:., totalPrice:}, {orderId:2, Date:., totalPrice:},..., {orderId:32, Date:., totalPrice:}],

}

Order{

_id:

custID:

date:

custFirstName:

CustLastName:

...,

ProductList:[{Name:., Description:., Category:., Quantity:., ...}, {Name:., Description:., Category:., Quantity:., ...}],

Status:

TotalPrice:

}

Product{

_id:.

Name:.

Category:.

shortDescription:.

```
price;,
--:,
--:,
type: { _id:, name:,, },
}
```

Esempio 17 – Relazioni one to many e many to many document DB

- Relazione “One to many”:

```
{
  customer_id: 76123,
  name: 'Acme Data Modeling Services',
  person_or_business: 'business',
  address : [
    { street: '276 North Amber St'
      city: 'Vancouver',
      state: 'WA',
      zip: 99076} ,
    { street: '89 Morton St',
      city: 'Salem',
      state: 'NH',
      zip: 01097}
  ]
}
```

Consideriamo l'entità customer che ha associata una lista di entità “Address”. La “one entity” nella relazione “one to many” è il documento primario, il customer in questo caso, mentre le “many entities” sono rappresentate come un array di documenti embedded (si usa quindi il document embedding), che sono documenti Address.

- Relazione “many to many”:

```
{
  { courseID: 'C1667',
    title: 'Introduction to Anthropology',
    instructor: 'Dr. Margret Austin',
    credits: 3,
    enrolledStudents: ['S1837', 'S3737', 'S9825' ...
      'S1847'] },
  { courseID: 'C2873',
    title: 'Algorithms and Data Structures',
    instructor: 'Dr. Susan Johnson',
    credits: 3,
    enrolledStudents: ['S1837', 'S3737', 'S4321', 'S9825'
      ... 'S1847'] },
  { courseID: C3876,
    title: 'Macroeconomics',
    instructor: 'Dr. James Schulen',
    credits: 3,
    enrolledStudents: ['S1837', 'S4321', 'S1470', 'S9825'
      'S1847'] }
}

{
  {studentID: 'S1837',
    name: 'Brian Nelson',
    gradYear: 2018,
    courses: ['C1667', 'C2873', 'C3876']},
  {studentID: 'S3737',
    name: 'Yolanda Deltor',
    gradYear: 2017,
    courses: [ 'C1667', 'C2873']},
  ...
}
```

Consideriamo questo esempio dove: uno studente può essere iscritto in più corsi e un corso può avere più studenti iscritti. Il problema può essere modellato con due collezioni come in

figura sopra. Bisogna fare attenzione quando si fanno gli aggiornamenti in questo tipo di relazioni. Infatti, il DBMS non controlla la “referential integrity” come in quello relazionale.